



13

EVADING ENDPOINT AND NETWORK DEFENSES



To successfully infiltrate and operate in its target environment, modern malware must survive that environment's defenses. Many targets, especially those in an enterprise environment, are behind multiple defensive applications and products that work tirelessly to protect the systems and networks that make up the organization's infrastructure. Malware can take active measures to evade these defenses (for example, by tampering with host defense applications) or take a passive approach to try to slip by them undetected.

In this chapter, I'll outline different types of defenses that malware could encounter on victim hosts and networks, and then I'll explain some techniques it might employ to get around those defenses. Defense evasion is a huge topic, so this chapter will focus primarily on the most common tactics.

An Endpoint Defense Primer

The endpoint defense market is flooded with overloaded terms and fancy product names. It can be difficult to keep track of it all and understand at a fundamental level what each tool actually does. In this section, I'll try to establish a common vocabulary for endpoint defense tools. First, though, we'll go over a brief history of endpoint defenses to establish a baseline level of knowledge on which we can build.

A Brief History of Endpoint Defense Technology

Endpoint defense software goes back to the early days of malware, the 1970s and 1980s. At that time, endpoint defenses were appropriately called *antivirus (AV)* products, as most types of malicious software were widely known as *viruses*. In those early days, malware was relatively simple and AV software simply searched files for specific malicious patterns. Those AV programs were typically developed by hobbyists.

In the late 1980s and early 1990s, malware started to become more sophisticated, and commercial AV software companies (including Norton, Kaspersky, and McAfee) emerged to address the growing threat. The software these companies developed provided more advanced features than the hobbyist AV programs, such as real-time scanning, heuristics-based detection, and automatic signature updates.

As malware authors were forced to adapt to technological advances, they created even more complex and covert malware capable of evading traditional AV software. In response, AV vendors started to develop much more robust software (which I'll refer to as *anti-malware software* going forward), as well as products called *endpoint protection platforms (EPPs)*. EPPs are a more complete host protection solution that includes features not found in traditional anti-malware, such as built-in software firewalls and host intrusion prevention. More recently, *endpoint detection and response (EDR)* solutions have emerged as a more advanced endpoint security solution that provides real-time visibility into endpoint activity and enables security teams to quickly detect and respond to security threats.

There's a significant amount of overlap among host defense technologies. Many modern anti-malware products, for example, incorporate aspects of EPPs and even EDR solutions. EPPs include all of the functionalities of traditional and modern anti-malware and may overlap with some EDR solutions. EDR typically includes all of the functionalities of anti-malware and EPPs, as well as others. Thus, for the sake of simplicity, I'll categorize endpoint defenses as either anti-malware or EDR in this book. Note, however, that each product vendor has its own "secret sauce," so these categories are meant to reflect how the defenses work in general, not the specific details of how they operate in practice.

Anti-malware

Perhaps the most common and well-known host defense technology is *anti-malware*, or what was formerly called *antivirus*. It specializes in detecting and identifying malware threats on the system, both on the hard disk (before malware executes itself) and in memory (after the malware

is running on the system). To accomplish this, anti-malware software uses a range of techniques, including hash-based and signature-based detection as well as heuristic and behavioral analysis.

Hash-Based Detection

Hash-based detection is a primitive method used in early anti-malware scanners, and it's still used today to some extent. Anti-malware vendors maintain a database of file hashes known to be either benign or malicious. When a file is written to disk, the anti-malware software scans the file and compares its hash to the database. If the file is known to be benign, the anti-malware software leaves it alone. If the file is known to be malicious, the anti-malware software automatically removes it from the system and places it into a special quarantine where it can do no harm.

The primary problem with this hash-based method is that the files must already be known to the anti-malware software. If the file is new and not in the database, the anti-malware can't detect it (at least, not using hash analysis alone). According to a December 15, 2020, press release from Kaspersky, about 360,000 new malicious files were created every day that year. This obviously poses a problem for hash-based detection methods, since anti-malware software can't possibly keep up.

Signature-Based Detection

Signature-based detection, an upgrade to the older hash-based detection methods, uses *signatures*, or known patterns, to identify malicious code in files or process memory. These patterns can be strings, byte sequences, metadata, or anything else that indicates that the file or memory segment could be related to malware. Anti-malware software maintains a large database of signatures, and when it identifies a match on one, it raises an alert and quarantines the respective file or terminates the suspect process. Signature-based detection functions similarly to the Yara rules discussed in [Chapter 2](#).

Over time, malware authors have caught on to these detection techniques. For example, since signature-based detection is looking for malicious patterns, malware authors can simply encrypt or obfuscate their malware on disk or in memory to hide or change these patterns. Worse, they can generate countless variations of their malware so that, once again, the detection mechanisms can't keep pace. It's clear that anti-malware software has had to evolve.

Heuristic-Based Detection

Instead of matching file hashes or looking for specific patterns in a piece of software, *heuristic-based detection* looks at a file's behavior. It does this by using a few different sub-techniques. First, it inspects the file for signs of malicious code. This goes beyond simple string and byte-sequence pattern matching, however; it's looking for indicators such as suspicious blocks of assembly instructions or atypical uses of API calls. This process may also involve a weighting or scoring system. When the anti-malware software discovers a suspicious sequence of code, the file's score increases. Once this score hits a predefined threshold, the anti-malware engine rates the file as malicious.

More modern heuristic-based methods can also employ file emulation techniques to better understand the file. *File emulation* involves executing the file in an *emulation engine*, which is a very lightweight virtual machine that can dynamically assess the file before it actually executes on the system. As CPU instructions are executed in the emulator, the anti-malware software monitors the file's behaviors, and if any are suspicious or raise the file's score high enough, the anti-malware takes further action.

Cloud-Based Analysis Sandboxes

Running each file on a system through an emulation engine would be very taxing, so some modern anti-malware vendors instead use cloud-based analysis sandboxes. If a file is still unclassified even after it's been subjected to hash-based, signature-based, and heuristic-based detection mechanisms, the file will be sent to the anti-malware vendor's cloud environment for sandboxing and further analysis. *Cloud-based sandboxing* is a form of crowdsourced security in that all customers who use such anti-malware software will be notified if a particular file is malicious, even if they've never seen it.

Figure 13-1 summarizes the detection mechanisms we've covered so far.

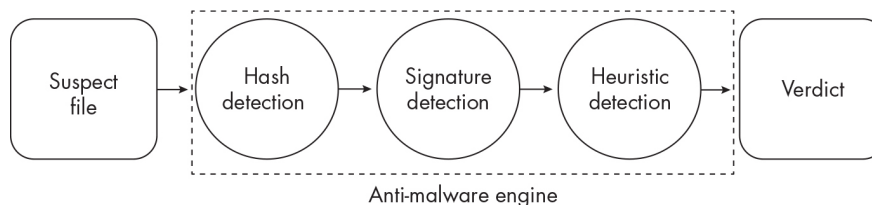


Figure 13-1: Typical detection mechanisms in anti-malware software

Note that many modern anti-malware solutions use a combination of some or all of these techniques.

Limitations and Challenges

Even though anti-malware software has progressed and improved substantially over time, it still has some limitations. As a whole, it's effective at identifying and eliminating threats from files that are known to be malicious; those that share similarities with files that are known to be malicious; and those that aren't especially advanced, targeted, or bespoke. More advanced and specific threats can sneak past anti-malware software (as we'll see later in this chapter).

Another limitation of anti-malware software is that it must be mindful of system resources. Anti-malware software has a very difficult job: it must scan potentially thousands of files and memory regions at a time while keeping the lowest system resource footprint possible to avoid disrupting the end-user experience. This means that it can't run every file through the emulation or sandboxing process; it must reserve those more time- and resource-intensive techniques for suspicious files that require deeper investigation.

Even among the files it deems malicious, anti-malware software must be selective. For example, anti-malware software has been known to flag critical system files as malicious, an action that cannot be tolerated. As a result, anti-malware products may lean toward giving a file a pass rather than flagging it as malicious.

One final limitation of anti-malware software is that while it's intended to detect and eradicate threats on the endpoint, it wasn't designed with post-compromise investigations or context in mind. Modern and advanced attacks often involve a number of steps in the attack chain that use multiple techniques and components, and anti-malware alone can leave blind spots, especially in complex corporate environments. This is where EDR comes into play.

Endpoint Detection and Response

Endpoint detection and response (EDR) solutions provide more advanced threat detection and response features than traditional anti-malware solutions provide. While anti-malware solutions are focused primarily on detecting and defending against known malware threats, EDR solutions are able to detect and act upon a broader range of advanced threats.

One of EDR's primary benefits is its ability to establish context around an attack. It creates this context, often referred to as *telemetry*, by collecting data from multiple endpoints, enabling investigators to perform deeper analysis and to identify similar patterns of malicious activity across the

enterprise. EDR can even help investigation teams identify the “patient zero” of an attack.

Let’s take a quick look under the hood to see how EDR works. Typically, EDR consists of multiple components in both user space and kernel space, plus a log aggregator and analysis engine. We’ll start with the user-space components.

User-Space Components

EDR solutions always have at least one executable running as a process in user space. This process, often called an *agent*, runs in a high-privilege context and monitors other processes on the host for suspicious behaviors, intervening when necessary. To do this, the EDR agent collects and analyzes system events, then forwards this information to a log aggregator, which we’ll discuss shortly. When a new process is created on the host, the EDR process can inject a hooking module into it using various methods such as the ones discussed in [Chapter 12](#).

[Figure 13-2](#) illustrates malware being hooked by EDR.

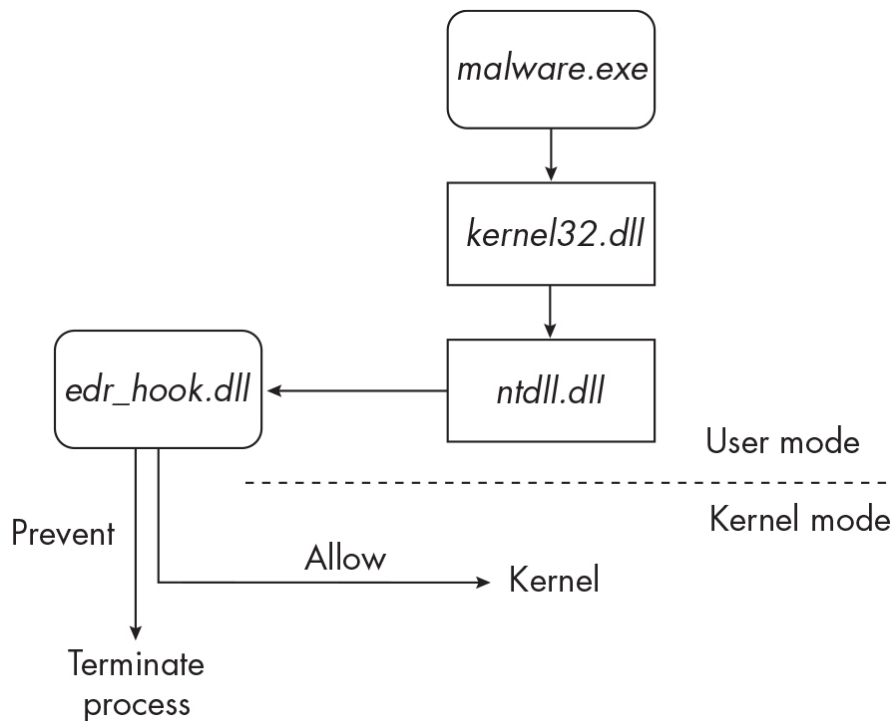


Figure 13-2: The EDR process using a module to hook malware

The malware executable (*malware.exe*) is calling a WinAPI function residing in *kernel32.dll*, which subsequently invokes *ntdll.dll*. Instead of *ntdll.dll* making a syscall into kernel space (which is what would happen under normal circumstances), the installed EDR software is hooking *ntdll.dll* via a previously injected DLL (*edr_hook.dll*). This allows the EDR

software to decide whether to permit the API call or to block or even terminate it. If the EDR software deems the API activity benign, it allows the API call to continue to the kernel as normal.

Some common functions that may be hooked by modern anti-malware and EDR software include the following:

- Memory operations (such as `NtProtectVirtualMemory` and `NtReadVirtualMemory`) that monitor memory commits and protection changes, such as when the EDR software wishes to know whether a region of memory is changed to *read-write-executable (RWX)*, indicating that there's potentially code in this region about to be executed
- Functions that create and terminate processes (such as `NtCreateProcessEx`, `NtCreateUserProcess`, and `NtTerminateProcess`) so that the EDR software can watch over and hook into newly created processes
- Functions that load libraries (such as `LdrLoadDll`) so that the EDR software can monitor suspicious processes as they load new libraries and modules
- Functions commonly used for process injection (such as `NtOpenProcess`, `NtCreateThread`, `NtResumeThread`, `NtUnmapViewOfSection`, and `NtMapViewOfSection`) so that the EDR software can monitor for code- and hook- injection attempts
- File writes (such as `NtWriteFile`), which are often used by ransomware and other destructive malware
- Functions that attempt to create network connections, such as `InternetOpen` and `InternetConnect`, which EDR can monitor for suspicious network communication

Hooking API function calls is only one way of monitoring the system. Some EDR products also collect system data using other sources, such as *Event Tracing for Windows (ETW)*, a mechanism designed for logging and diagnostics that's been a part of Windows since 2007. ETW is able to collect and log data from many sources, including user-space processes and kernel drivers, which makes it quite useful for EDR ingestion as well. You can read more about it in the Microsoft documentation.

To identify suspicious activity, EDR agents might rely upon a threat-scoring system. To illustrate this, consider the attack scenario shown in **Figure 13-3**.

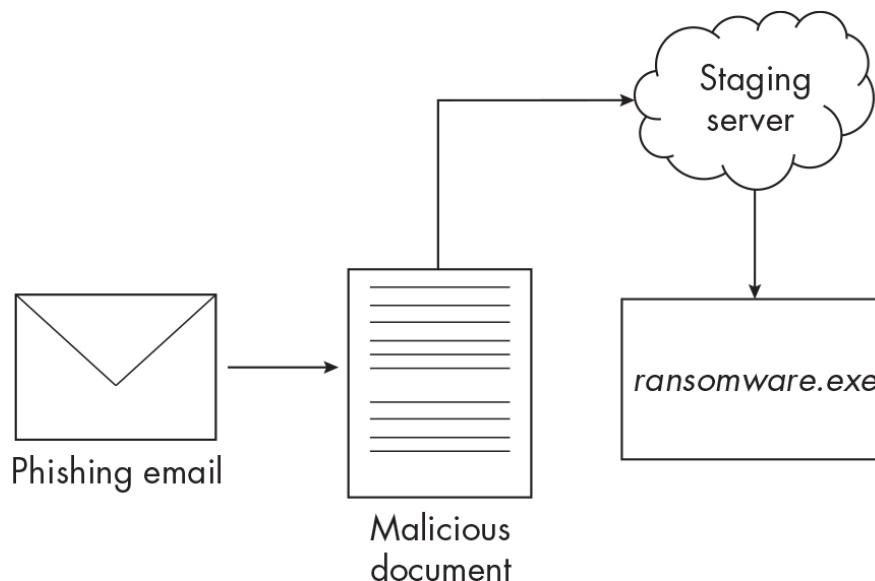


Figure 13-3: A multistage attack

First, the attacker sends a target user a specially crafted phishing email containing a malicious Microsoft Office document attachment. The Office document's malicious code executes PowerShell to contact a staging server where the attacker's malware payload (*ransomware.exe*) is hosted, then downloads and executes the payload in memory. The ransomware payload subsequently begins encrypting files on the victim's hard disk.

The EDR product assigns a score to each event that makes up this attack. Say it assigns a threat value of 35 to the Office document that launched a PowerShell command, a value of 20 to the connection to the suspicious server, and a value of 30 to the download and execution of an unknown executable file. If the maximum threat score is a hypothetical 100, and if any value over 75 is considered malicious, then this series of events totaling 85 would trigger the EDR solution to perform actions such as killing the Office and PowerShell processes or terminating the network connections to the remote server. The ransomware execution itself would likely have a high threat score, as ransomware creates multitudes of file write actions. Furthermore, metadata related to these events would be forwarded to a central log storage and processor, allowing the EDR product to analyze this attack in the context of the entire enterprise infrastructure.

Keep in mind that this threat-scoring system example is intentionally simplified and merely intended to demonstrate how EDR is better equipped than traditional endpoint defenses to “connect the dots” during an advanced attack. Let's move on to another important part of EDR solutions: kernel-space components.

Kernel-Space Components

In kernel space, EDR solutions primarily take the form of kernel drivers or modules, which are pieces of compiled code that execute in kernel address space. EDR kernel drivers rely on *callbacks*, functions registered by kernel components to receive notifications in response to specific events. To monitor the system for new processes, for example, the EDR kernel driver registers a callback routine by invoking the kernel function `PsSetCreateProcessNotifyRoutine`. When a process is created or terminated, the EDR kernel driver will be notified so that it can respond to that event accordingly, likely by engaging its user-space component to inject hooks into the newly created process to begin monitoring. **Figure 13-4** illustrates how this works.

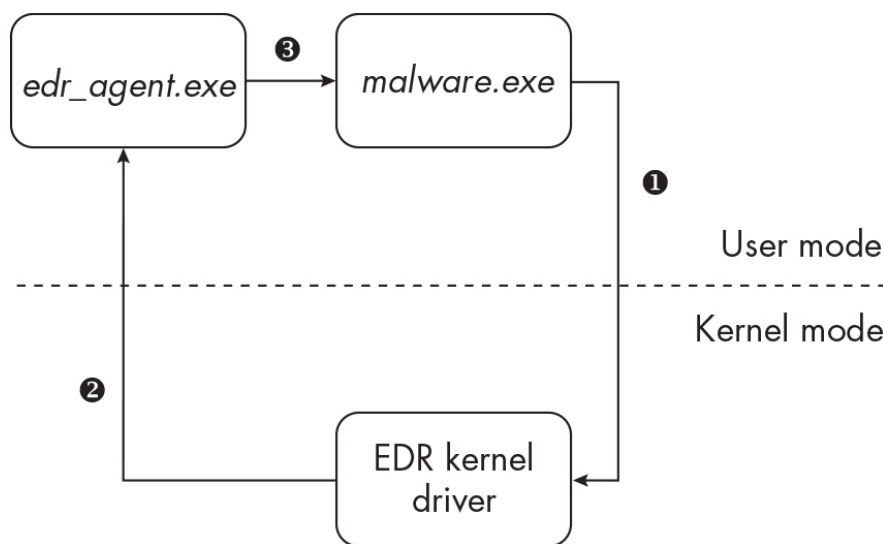


Figure 13-4: EDR hook injection

In this example, the EDR kernel driver is notified via `PsSetCreateProcessNotifyRoutine` (not shown) that a new process, `malware.exe`, is being created **1**. The EDR driver instructs the EDR agent in user space (`edr_agent.exe`) **2** to inject a hook into the new process **3**. Now `malware.exe` is being monitored by the EDR.

Here are a few other callbacks that EDR software might use, plus the circumstances in which they're triggered:

`PsSetCreateThreadNotifyRoutine` This is triggered when any new thread is created or deleted.

`PsSetLoadImageNotifyRoutine(Ex)` This is triggered when a process loads an image (such as a DLL module) into memory, or when a new driver is loaded.

IoWMISetNotificationCallback This is triggered by Windows Management Instrumentation (WMI) events.

CmRegisterCallback(Ex) This is triggered when any running thread modifies the Windows registry.

FsRtlRegisterFileSystemFilterCallbacks This is triggered when certain filesystem operations occur.

IoRegisterBootDriverCallback This is triggered when a new boot-start driver is initialized. Boot-start drivers start on system boot-up, so this callback can be used by anti-malware and EDR to detect rootkits and bootkits that use a boot-up driver (more on rootkits in [Chapter 14](#)).

EDR can also take advantage of minifilter drivers. *Minifilters* are used to monitor requests to the filesystem from user-space processes, so EDR can use them to intercept and block malicious filesystem actions that malware is trying to execute (for example, ransomware opening and writing to files at a rapid rate). Another reason EDR uses minifilters is to monitor and protect its own files from tampering or deletion. In the event that a malware sample is able to establish low-level privileges and attempt to remove or modify EDR components, the minifilter drivers will notify the EDR product about this activity.

Other kernel components that EDR may use are network filter drivers and *early-launch anti-malware (ELAM)* drivers. Network filter drivers can be used to monitor, intercept, and modify network communication, which can be useful in detecting anomalous network traffic such as command and control (C2) traffic. ELAM drivers load before the operating system boots and help protect against malware that could tamper with the boot process.

NOTE

You may have noticed that EDR injects modules into other processes, installs hooks to intercept API calls, and installs drivers into kernel space, which is eerily similar to malware! EDR does indeed look suspiciously similar to a variant of malware called rootkits. We'll talk about rootkit techniques and components in more detail in [Chapter 14](#).

Logging and Analysis

I mentioned previously that one of the EDR agent's responsibilities is to forward events and associated data to a central logging server, where they can be further analyzed and stored for future investigations. [Figure 13-5](#) illustrates this process.

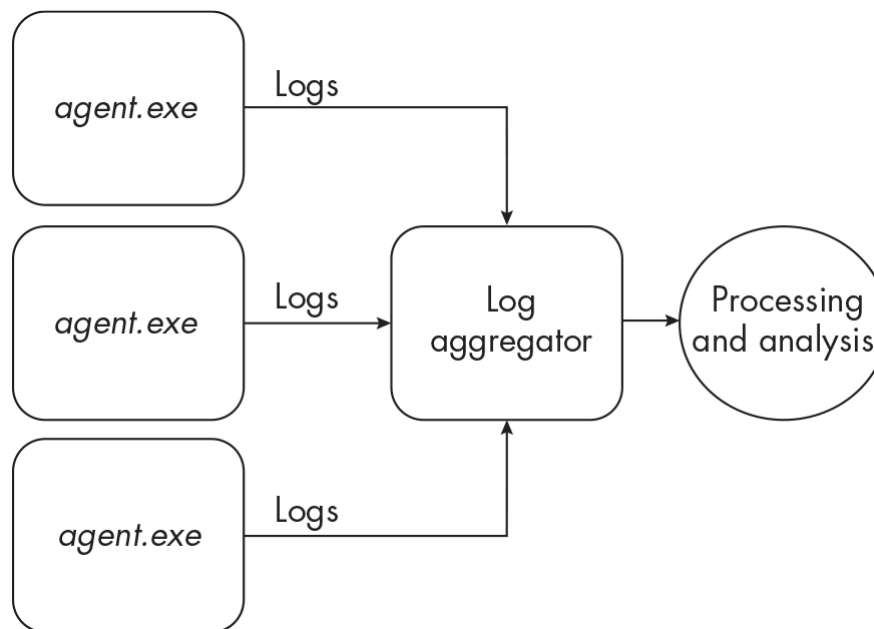


Figure 13-5: An EDR agent gathering data from multiple endpoints for analysis

As you can see here, EDR agents running on disparate endpoints in the enterprise infrastructure forward data to the EDR log aggregator. Using various vendor-specific techniques, the data is processed and analyzed, enabling the EDR product to detect larger-scale attacks, for example. Using this telemetry, some EDR products can even “learn” what’s normal for this specific enterprise environment, helping them further differentiate between typical end-user behaviors and malicious activity. Many modern EDR solutions can also collect logs from many different sources to monitor an organization’s entire IT infrastructure. These EDR solutions are sometimes referred to as *extended detection and response (XDR)* platforms. XDRs are able to collect and analyze data from sources such as client endpoints, servers, network devices, applications, and cloud computing environments. For the remainder of this book, I’ll use the term *EDR* to also refer to XDR.

Even with all its benefits and capabilities, EDR can be (and sometimes is) bypassed by advanced malware. Now that you have a basic understanding of how EDR and anti-malware software work, let’s look at how malware can tamper with and circumvent these defenses. Advanced malware will often first attempt to understand the environment in which it is operating; this involves the malware scouting the target system and network and attempting to identify the endpoint defenses standing in its path. We’ll start there.

Identifying Endpoint Defenses

Since many anti-malware and EDR products behave and monitor the system a bit differently, an advanced attacker will conduct at least basic re-

connaissance on the target prior to executing a malware payload. There are a number of techniques an attacker might employ to enumerate the target's defenses. For example, the following PowerShell command will retrieve a list of installed anti-malware software on the host:

```
PS C:\> Get-CimInstance -Namespace root/SecurityCenter2 -ClassName AntivirusProduct
```

Here's some example output for this command:

```
displayName           : Windows Defender
instanceGuid          : {D68BAD3A-821F-4fce-9E54-DA133F2CBA26}
pathToSignedProductExe : windowsdefender://
pathToSignedReportingExe : %ProgramFiles%\Windows Defender\MsMpEng.exe
productState          : 397568
timestamp              : Mon, 13 Feb 2023 18:29:52 GMT
--snip--
```

On my system, Microsoft Defender was identified as the active anti-malware product. (This output uses the outdated name *Windows Defender*; the product has since been renamed Microsoft Defender.)

Although this command will retrieve only anti-malware software that has registered itself within the Microsoft Security Center, most anti-malware vendors do this. A more robust solution is to enumerate artifacts on the host, searching for signs of endpoint defense products.

In [Chapter 4](#), we discussed how malware enumerates and lists processes running on the host for sandbox and VM detection. It can do the same for enumerating defenses. For example, it can call the Windows API functions `CreateToolhelp32Snapshot` to create a “snapshot” of all processes running on the victim host, then iterate through these processes using `Process32First` and `Process32Next`. It can then search for suspect process names such as *Msmpeg.exe* (the default Windows 10 and 11 anti-malware process), *AvastSvc.exe* (the executable process for the anti-malware product from Avast), or any process with the string `falcon` in its name (which could be related to the CrowdStrike Falcon EDR product) in order to determine what kinds of defenses are on the host.

Some of the most popular anti-malware and EDR vendors and products include the following:

- CrowdStrike Falcon EDR: *CSFalcon*.exe*
- ESET Smart Security: *Ekrn.exe*
- Kaspersky: *Avp.exe*
- Malwarebytes: *Mbam.exe*

- McAfee Endpoint Security: *masvc.exe*
- Sophos: *SavService.exe*
- Symantec Endpoint Protection: *ccSvcHst.exe*
- VMware Carbon Black: *cb.exe*
- Microsoft Defender for Endpoint (formerly Windows Defender):
MsMpEng.exe

These processes often have associated services, which are named similarly and can be enumerated by malware. Also keep in mind that the list is not exhaustive and is subject to change between software versions.

Alternatively, scanning the host for anti-malware and EDR-related files, drivers, and registry keys can be effective. **Figure 13-6** shows the Procmon output of a malware sample that queries the registry for keys that match common anti-malware vendors.



The screenshot shows a list of registry queries performed by a malware sample. Each entry consists of an icon, an operation (RegQueryValue), a path, and a value name. The paths are primarily under HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run and HKLM\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Run. The values are names of various anti-malware and EDR products.

Operation	Path	Value Name
RegQueryValue	HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run	AVG_UI
RegQueryValue	HKLM\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Run	AVG_UI
RegQueryValue	HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run	AVP
RegQueryValue	HKLM\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Run	AVP
RegQueryValue	HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run	mcui_exe
RegQueryValue	HKLM\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Run	mcui_exe
RegQueryValue	HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run	mcplui_exe
RegQueryValue	HKLM\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Run	mcplui_exe
RegQueryValue	HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run	Bdagent
RegQueryValue	HKLM\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Run	Bdagent
RegQueryValue	HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run	Trend Micro Titanium
RegQueryValue	HKLM\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Run	Trend Micro Titanium
RegQueryValue	HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run	Trend Micro Client Framework
RegQueryValue	HKLM\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Run	Trend Micro Client Framework
RegQueryValue	HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run	avast
RegQueryValue	HKLM\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Run	avast
RegQueryValue	HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run	MSC
RegQueryValue	HKLM\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Run	MSC
RegQueryValue	HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run	BullGuard
RegQueryValue	HKLM\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Run	BullGuard
RegQueryValue	HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run	Sophos AutoUpdate Monitor
RegQueryValue	HKLM\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Run	Sophos AutoUpdate Monitor
RegQueryValue	HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run	SpIDerAgent

Figure 13-6: Malware querying the registry for names of anti-malware and EDR products

Similarly, the anti-malware product from Kaspersky, for example, installs itself to a directory such as *C:\Program Files (x86)\Kaspersky Lab\Kaspersky Total Security* and installs several drivers on the host that are visible in the *C:\System32\drivers* directory, which could alert malware to its presence (see **Figure 13-7**).

This PC > Local Disk (C:) > Windows > System32 > drivers >

Name	Date modified	Type
klpd.sys	2/16/2022 5:47 PM	System file
klpnflt.sys	2/16/2022 5:47 PM	System file
klwfp.sys	2/16/2022 5:47 PM	System file
klwtp.sys	2/16/2022 5:47 PM	System file
kneps.sys	2/16/2022 5:47 PM	System file
klkbdflt.sys	2/16/2022 5:47 PM	System file
klmouflt.sys	2/16/2022 5:47 PM	System file
klupd_klif_arkmon.sys	3/24/2022 5:09 AM	System file
klupd_klif_klbg.sys	3/24/2022 5:09 AM	System file
klupd_klif_mark.sys	3/24/2022 5:09 AM	System file
klupd_klif_klark.sys	3/24/2022 5:11 AM	System file

Figure 13-7: Kaspersky's installed drivers

There are a number of research projects, such as at <https://github.com/ethereal-vx/Antivirus-Artifacts>, that catalog the different artifacts that endpoint defense products create on the host. These resources can be useful to both malware researchers and malware authors.

Finally, malware already running on the host may attempt to look for signs that it is being hooked by anti-malware or EDR. For instance, the malware might enumerate its loaded modules using `Module32First` and `Module32Next` (as described in [Chapter 7](#)) or search its process memory for signs of hooking, as it does to circumvent sandbox hooking (described in [Chapter 8](#)). These hooks may reveal a specific defense product.

Once an endpoint security product has been detected and identified, the malware can take one of several active or passive approaches to circumvent it. First, we'll discuss some active measures.

Actively Circumventing Endpoint Defenses

Active circumvention is directly bypassing defenses by disabling them completely, modifying policies to weaken them, or tampering with them to blind them. There are many ways to accomplish this, so let's look at some examples.

Disabling Host Defenses

A crude but effective method for malware to evade and bypass endpoint security is simply to disable it. After enumerating the security products running on the endpoint, malware can disable them in several ways. First, it can attempt to kill the application's processes (examples include anti-malware and EDR-related processes, Windows Security Center processes and services, and any firewall products such as the Windows built-

in host firewall) by calling the `TerminateProcess` function or invoking tools such as PowerShell or the Windows command line.

Some anti-malware solutions store parts of their configurations in the registry; tampering with these can also produce unwanted results. The malware family KillAV (SHA256:

659ce17fd9d4c6aad952bc5c0ae93a748178e53f8d60e45ba1d0c15632fd3e3f), which specializes in doing what its name suggests, attempts to disable the default Windows anti-malware and threat protection services by stopping processes and tampering with the registry. Here are some of the commands that this malware executes:

```
net stop WinDefend

REG ADD "HKLM\Software\Policies\Microsoft\Windows Defender\Real-Time Protection" /v "DisableRealtimeMonitoring" /t REG_DWORD /d "1" /f

REG ADD "HKLM\Software\Policies\Microsoft\Windows Defender" /v "DisableAntiSpyware" /t REG_DWORD /d "1" /f

REG ADD "HKLM\SYSTEM\CurrentControlSet\Services\WdBoot" /v Start /t REG_DWORD /d 4 /f
```

The `net stop WinDefend` command terminates the Microsoft Defender process, while the various `REG ADD` commands tamper with the registry to switch off the Microsoft Defender real-time monitoring service, the AntiSpyware service, and the AV boot driver (WdBoot), all of which are critical components of the Windows default anti-malware services.

Some malware simply attempts to uninstall defense software from the host, using PowerShell commands like the following:

```
PS C:\> $antimalware = Get-WmiObject -Class Win32_Product | Where-Object{$_.Name -eq "ES  
Endpoint Antivirus"}  
PS C:\> $antimalware.Uninstall()
```

The first PowerShell command leverages WMI to enumerate installed software on the host, looking specifically for ESET anti-malware software; the second command uninstalls it.

A possibly less noisy approach is lowering an endpoint defense process's *priority*: that is, reducing how important the process is to the operating system. Higher-priority processes (typically, critical processes such as system processes) are allotted more CPU time. By lowering an endpoint defense process's priority using `SetPriorityClass`, the malware can diminish its effectiveness. The following pseudocode demonstrates this approach:


```
// Open the target process (anti-malware process, for example).  
hProcess = OpenProcess(..., ..., TargetProcessPID);  
  
// Set process priority to "Low".  
SetPriorityClass(hProcess, BELOW_NORMAL_PRIORITY_CLASS);
```

There is a catch with these techniques, however: depending on the operating environment, the malware may require high-level privileges to terminate anti-malware processes and services, tamper with registry configurations and process priorities, and uninstall software. In this case, the malware must elevate its privileges before trying to disable endpoint defenses. We'll look at how it does so in the "Privilege Elevation for Defense Evasion" section on [page 248](#).

Adding Anti-malware Exclusions

Another way to impair defenses is by tampering with the anti-malware exclusion list. Anti-malware software regularly scans specific filesystem directories on the endpoint, which are determined by how the user or organization configures the software. Most anti-malware software allows users to add exclusions, in the form of directory paths, to its configuration settings. Any files or directories in this exclusion list won't be monitored or scanned.

To employ this technique, an initial dropper or loader malware creates an exclusion on the victim host, and once that exclusion is enabled, it deploys the payload into the excluded directory. Here's how malware could use PowerShell to create an exclusion in the Microsoft Defender anti-malware solution:

```
PS C:\> Add-MpPreference -ExclusionPath "C:\Malware\DoNotScanThisDirectory"
```

This command adds an anti-malware exclusion for the path `C:\Malware\DoNotScanThisDirectory\`.

Note that the malware must have high-level privileges on the victim host to create exclusions in later versions of Windows, making the barrier to entry for this kind of attack more difficult than in older versions of the operating system.

Disabling Other Security Controls

In addition to disabling anti-malware, EDR, and other defenses, malware can disable other security features of the system. While such security fea-

tures may not directly prevent attacks, disabling them can degrade the overall security of the host, making it more susceptible to further attacks.

For example, malware may disable the Windows Update service, which is responsible for periodically updating Windows to patch bugs and vulnerabilities. If a threat actor disabled this service under the radar, it could ultimately degrade the overall security of the host over a longer period of time, leaving the system open to follow-up attacks. To disable a service with PowerShell, the malware could execute the following command:

```
PS C:\> Stop-Service "Service_Name"
```

The malware could also disable PowerShell security. Some Windows environments prohibit the execution of unauthorized PowerShell scripts, so switching off script execution controls could help an attacker execute PowerShell scripts they otherwise couldn't. To enable PowerShell script execution, an attacker can issue the following command:

```
PS C:\> Set-ExecutionPolicy Unrestricted
```

To prevent unauthorized outbound traffic from an endpoint, it's common to configure a host-based firewall solution. Host-based firewalls can be used to allow outbound traffic from specific processes, such as web browsers, while preventing outbound traffic from processes that shouldn't be communicating with any other hosts on the network or the internet. To get around this limitation, an attacker can directly modify firewall configurations on the host. This configuration differs depending on the firewall software being used, but for the standard Windows firewall, a rule can be added or modified with PowerShell. For example, malware could use the following command to change the default firewall policy to permit all outbound traffic from all processes on the host:

```
PS C:\> Set-NetFirewallProfile -Name Domain -DefaultOutboundAction Allow
```

Malware can also disable non-security-related tools that could be used to expose it. Examples include terminating and disabling the Windows Task Manager to prevent cautious users from spotting suspicious running processes or disabling the Registry Editor to prevent a knowledgeable system administrator from identifying malicious artifacts in the registry.

There are many policies and configurations on a Windows system that an attacker could alter to degrade security; these are just a few examples. The key takeaway is that a threat actor may not attempt to completely dis-

able endpoint defenses like anti-malware and EDR but may instead go for a less direct approach, making slight tweaks to the system to achieve its goals. This is a double-edged sword, however; the more the malware alters a system, the higher the chances it will be detected.

Blinding Defenses by Unhooking

Since EDR and modern anti-malware heavily rely on hooks to monitor suspicious processes and detect and prevent threats, unhooking techniques can create blind spots for them. The unhooking approaches outlined in the “Anti-hooking” section on [page 137](#) can be effective for some EDR and anti-malware software, but advanced host defenses are expecting them. These defenses may monitor their own hooks at the kernel level using their installed kernel components, and malware in turn could tamper with and unhook endpoint defenses at the kernel level.

The cat-and-mouse game that malware authors and host defenses play is evident here. Endpoint protection software monitors and hooks into malware. The malware scans for these hooks and unhooks them or tries to circumvent them in other ways. In response, the endpoint protection products check whether they’ve been unhooked, and the cycle continues. Endpoint defenses have some advantage here, however. If a program tries to remove hooks, the EDR or anti-malware software can assume that the process is malicious with some level of confidence, as there’s virtually no legitimate reason for unhooking.

As mentioned earlier in this chapter, endpoint defenses might use other sources of system monitoring, such as ETW, to supplement traditional hook-based monitoring. These data sources could also be blinded in various ways; in fact, there’s already a considerable amount of research on blinding ETW. This can be accomplished in multiple ways, one of which is to hook or patch `EtwEventWrite`, a function critical to ETW’s operation.

Exploiting Vulnerabilities in Host Defense Tooling

Anti-malware, EDR, and other defenses are developed by humans, and humans make mistakes, so inevitably there will be bugs in the product code that could lead to vulnerabilities threat actors can exploit. There haven’t been many publicly reported attempts to exploit defense tooling during attacks, but it’s always a possibility, especially for threat actors that have the means and ability to discover these bugs. A quick search in the MITRE CVE vulnerability database (<https://cve.mitre.org>) reveals some public vulnerabilities in anti-malware products, as shown in [Figure 13-8](#).

Search Results	
There are 273 CVE Records that match your search.	
Name	Description
CVE-2022-38611	Incorrect access control in Watchdog Anti-Virus v1.4.158 allows attackers to perform a DLL hijacking attack and execute arbitrary code via a crafted binary.
CVE-2022-27534	Kaspersky Anti-Virus products for home and Kaspersky Endpoint Security with antivirus databases released before 12 March 2022 had a bug in a data parsing module that potentially allowed an attacker to execute arbitrary code. The fix was delivered automatically. Credits: Georgy Zaytsev (Positive Technologies).
CVE-2021-44747	A Denial-of-Service (DoS) vulnerability was discovered in F-Secure Linux Security whereby the FmLib component used in certain F-Secure products can crash while scanning fuzzed files. The exploit can be triggered remotely by an attacker. A successful attack will result in Denial-of-Service of the Anti-Virus engine.
CVE-2021-40832	A Denial-of-Service (DoS) vulnerability was discovered in F-Secure Atlant whereby the AVRDL unpacking module component used in certain F-Secure products can crash while scanning a fuzzed files. The exploit can be triggered remotely by an attacker. A successful attack will result in Denial-of-Service (DoS) of the Anti-Virus engine.
CVE-2021-33603	A Denial-of-Service (DoS) vulnerability was discovered in F-Secure Atlant whereby the AVPACK module component used in certain F-Secure products can crash while scanning a fuzzed files. The exploit can be triggered remotely by an attacker. A successful attack will result in Denial-of-Service (DoS) of the Anti-Virus engine.

Figure 13-8: Vulnerabilities in anti-malware products, as reported by MITRE

This list includes vulnerabilities discovered in the anti-malware engines of Watchdog, Kaspersky, and F-Secure products. Most of the vulnerabilities listed are *denial-of-service (DoS)* bugs that could allow specially crafted code to crash the anti-malware engine or otherwise hamper its effectiveness.

That sums up our tour of active circumvention techniques. Now we'll turn to passive circumvention, which can be a stealthier and equally effective approach.

Passively Circumventing Endpoint Defenses

Passive circumvention techniques involve slipping past host defenses without directly tampering with them. As you'll see, these methods can be just as fruitful as active techniques without raising as many alarms.

Circumventing Monitoring

Chapter 8 discussed how malware can circumvent API hooking and monitoring in an attempt to evade sandboxes. Malware can use similar techniques to circumvent endpoint defenses. Because endpoint defenses also rely on function hooking to intercept and monitor suspicious activities, circumventing and bypassing these hooks is often an effective way to blind them.

Since anti-malware and EDR monitor various Windows API calls to detect malicious activities, one circumvention measure is to implement *direct syscalls*, or calls into kernel address space from user mode that circumvent the typical syscall procedure. Specifically, when a program invokes an API function in user mode, the operating system makes a syscall into *ntoskrnl.exe* to access the function's code (see **Figure 13-9**).

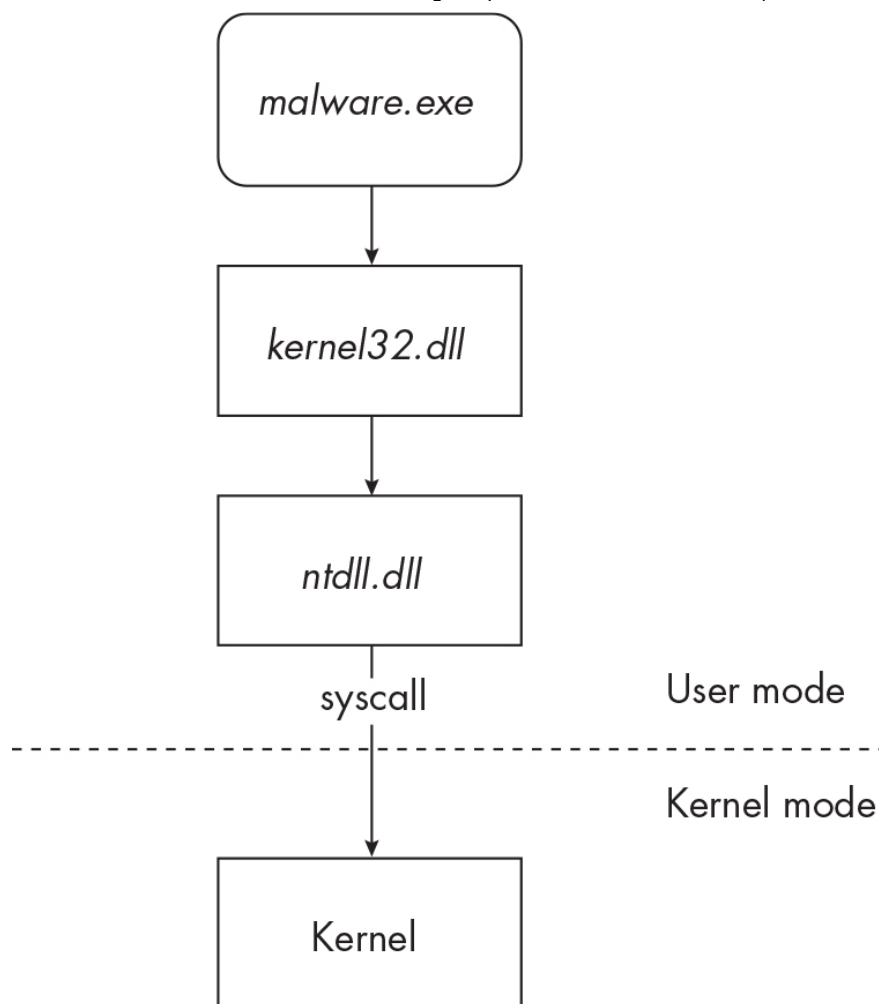


Figure 13-9: Normal syscall behavior

Instead of relying on the normal Windows and NT API calling process, though, malware can directly make the syscall (see [Figure 13-10](#)).

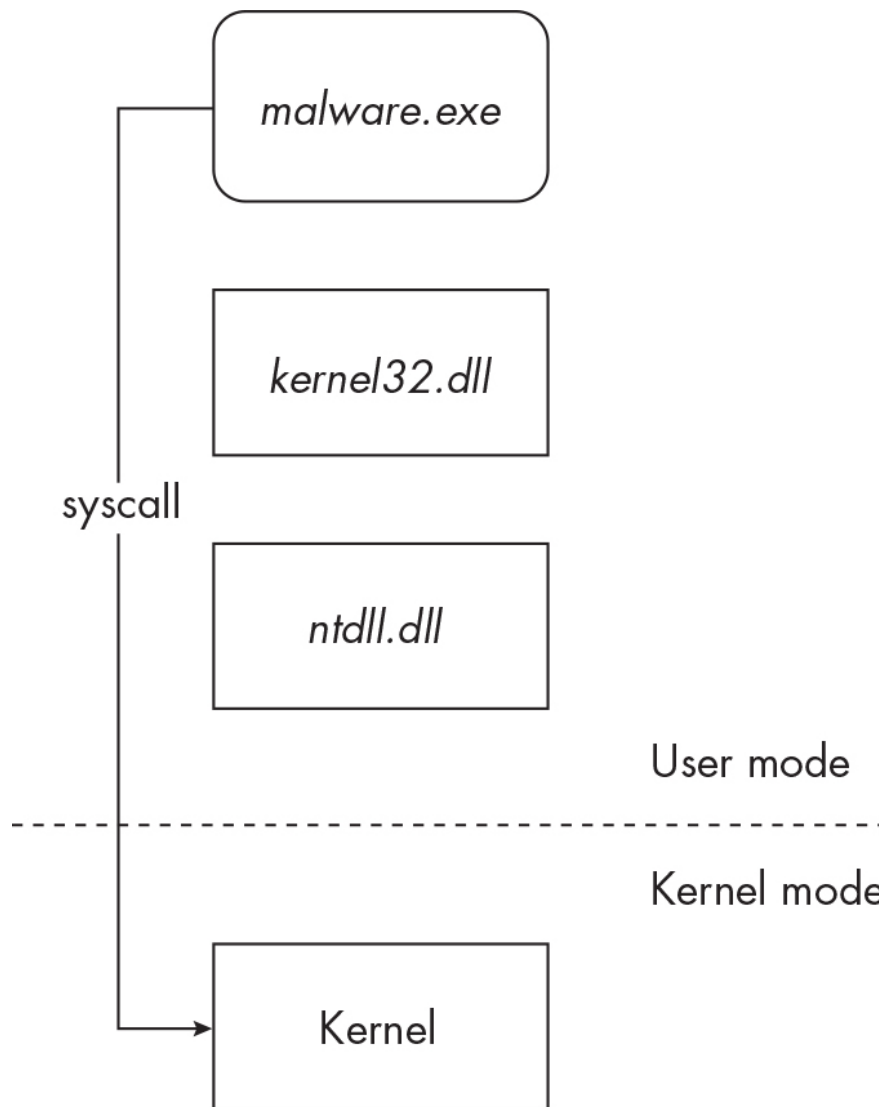


Figure 13-10: A direct syscall

Syscalls follow a basic pattern called a *syscall stub*. The purpose of the stub is to transfer execution flow from user mode to kernel mode, where the code of the function being invoked actually resides. The following code shows a syscall stub for the `NtAllocateVirtualMemory` function:

```
mov r10, rcx
mov eax, 18
test byte ptr ds:[7FFE0308], 1
jne ntdll.7FF80B57C3C5
syscall
ret
```

The most important part of the stub is the `mov eax, 18` line, in which 18 represents the *system service number (SSN)*, sometimes referred to as the *syscall number* or *syscall ID*. This number maps to the function code (in this case, `NtAllocateVirtualMemory`) that will be called in kernel mode. The next most important part of the stub is the `syscall` instruction (in hex, `0F`

05), which uses the SSN in the `eax` register to direct execution flow to the corresponding kernel function. The return (`ret`) instruction transfers execution flow back to the main program code once the syscall returns and has completed.

NOTE

The syscall stubs we've discussed thus far are specific to the x64 architecture. The x86 programs often use a different call: `sysenter`. `Sysenter` stubs are very similar:

```
mov edx, esp
```

```
sysenter
```

```
ret
```

The `mov edx, esp` instruction moves the SSN (stored in `ESP`) to `EDX` and passes it to the `sysenter` instruction (hex `0F 34`). Note that some x64 programs also use `sysenter` calls.

If malware wants to make a direct syscall (to avoid EDR or anti-malware hooks), it just needs to know the SSN of the function to call and then directly invoke the syscall. Sounds simple enough, but there are a few hurdles malware authors may face; most notably, function SSNs change depending on the Windows version and patch level of the target host. To get around this problem, malware authors have a few options. One option is to extract the SSNs from the `ntdll.dll` loaded in memory (see [Figure 13-11](#)).

00007FF80B57C550	4C:8BD1	mov r10,rcx	NtQueryInformationThread
00007FF80B57C553	B8 25000000	mov eax,25	25: '%'
00007FF80B57C558	F60425 0803FE7F 01	test byte ptr ds:[7FFE0308],1	
00007FF80B57C560	75 03	jne ntdll.7FF80B57C565	
00007FF80B57C562	OF05	syscall	
00007FF80B57C564	C3	ret	
00007FF80B57C565	CD 2E	int 2E	
00007FF80B57C567	C3	ret	
00007FF80B57C568	0F1F8400 00000000	nop dword ptr ds:[rax+rax],eax	
00007FF80B57C570	4C:8BD1	mov r10,rcx	ZwOpenProcess
00007FF80B57C573	B8 26000000	mov eax,26	26: '&'
00007FF80B57C578	F60425 0803FE7F 01	test byte ptr ds:[7FFE0308],1	
00007FF80B57C580	75 03	jne ntdll.7FF80B57C585	
00007FF80B57C582	OF05	syscall	
00007FF80B57C584	C3	ret	
00007FF80B57C585	CD 2E	int 2E	
00007FF80B57C587	C3	ret	
00007FF80B57C588	0F1F8400 00000000	nop dword ptr ds:[rax+rax],eax	
00007FF80B57C590	4C:8BD1	mov r10,rcx	NtSetInformationFile
00007FF80B57C593	B8 27000000	mov eax,27	27: ''
00007FF80B57C598	F60425 0803FE7F 01	test byte ptr ds:[7FFE0308],1	
00007FF80B57C5A0	75 03	jne ntdll.7FF80B57C5A5	
00007FF80B57C5A2	OF05	syscall	
00007FF80B57C5A4	C3	ret	
00007FF80B57C5A5	CD 2E	int 2E	
00007FF80B57C5A7	C3	ret	
00007FF80B57C5A8	0F1F8400 00000000	nop dword ptr ds:[rax+rax],eax	
00007FF80B57C5B0	4C:8BD1	mov r10,rcx	NtMapViewOfSection
00007FF80B57C5B3	B8 28000000	mov eax,28	28: '('
00007FF80B57C5B8	F60425 0803FE7F 01	test byte ptr ds:[7FFE0308],1	
00007FF80B57C5C0	75 03	jne ntdll.7FF80B57C5C5	
00007FF80B57C5C2	OF05	syscall	
00007FF80B57C5C4	C3	ret	

Figure 13-11: Syscall stubs in the loaded `ntdll.dll` module

Figure 13-11 illustrates what `ntdll.dll` syscall stubs look like loaded in memory. Notice the highlighted SSNs for some of the exported `ntdll.dll` functions. Malware can inspect its loaded modules in memory, locate the relevant syscall stubs, and identify the SSN it requires. Once it obtains the

SSN for the function it wants to call, it can craft its own syscall stubs, by-passing the standard syscall invocation from *ntdll.dll*. The following code illustrates the use of syscalls (more accurately, sysenters) in a variant of Remcos (SHA256:

45cd8dd797af4fd769eef00243134c46c38bd9e65e15d7bd2e9b834d5e8b3095):

```
--snip--
sub_1D29    proc near
    push    2BC12D2Eh
    call    resolve_ssn      ; NtMapViewOfSection
    call    invoke_sysenter
    retn    28h
--snip--
sub_1D3B    proc near
    push    3ACFFF22h
    call    resolve_ssn      ; NtWriteVirtualMemory
    call    invoke_sysenter
    retn    8
--snip--
sub_1D4D    proc near
    push    7C9B2C0Bh
    call    resolve_ssn      ; NtResumeThread
    call    invoke_sysenter
    retn    8
--snip--
```

This Remcos sample resolves the SSNs of the functions it wishes to call by enumerating the *ntdll.dll* file on disk (not shown here). It subsequently calls a function that contains the standard sysenter stub. Specifically, this Remcos sample is attempting to covertly inject code into another process by making direct sysenter calls to invoke the functions *NtMapViewOfSection*, *NtWriteVirtualMemory*, *NtResumeThread*, and others.

One problem with this approach is that these syscalls aren't originating from their intended module, *ntdll.dll*. This can be a dead giveaway for endpoint defenses looking for this type of anomaly. A stealthier option, sometimes called an *indirect* syscall, is for malware authors to simply re-use the syscalls in *ntdll.dll* rather than crafting their own syscall stubs. To accomplish this, the malware identifies the syscall stub address in *ntdll.dll* for the function it wishes to call. Then, rather than including a call to the *NtMapViewOfSection* function, for example, the malware implements a *jmp* instruction to jump directly to that syscall stub inside *ntdll.dll*.

These are just a few of the tricks malware authors might use to bypass hooks installed by endpoint defenses. However, these techniques are not foolproof. As you've seen, many can produce suspicious indicators (enumerating *ntdll.dll* memory, invoking syscalls directly from the malware's

code, making direct jumps into *ntdll.dll*, and so on). Not only that, but also many modern endpoint defenses hook into the kernel to attempt to spot this sort of activity.

Evasion using syscall invocation is a fascinating topic, so if you're interested in learning more, I recommend the following references:

- Usman Sikander, "AV/EDR Evasion Using Direct System Calls (User-Mode vs. Kernel-Mode)," *Medium*, March 11, 2022, <https://medium.com/@merasor07/av-edr-evasion-using-direct-system-calls-user-mode-vs-kernel-mode-fad2fded01a>.
- Cornelis, "Red Team Tactics: Combining Direct System Calls and sRDI to bypass AV/EDR," *Outflank*, June 19, 2019, <https://www.outflank.nl/blog/2019/06/19/red-team-tactics-combining-direct-system-calls-and-srdi-to-bypass-av-edr/>.
- Red Team Notes, "Calling Syscalls Directly from Visual Studio to Bypass AVs/EDRs," *Red Team Notes*, n.d., <https://www.ired.team/offensive-security/defense-evasion/using-syscalls-directly-from-visual-studio-to-bypass-avs-edrs>.

Circumventing Signature-Based Detection

As we've discussed, anti-malware uses signature-based detection mechanisms, meaning that it looks for common patterns to identify malicious code in files and in memory. Malware often contains strings, function names, or DLL and module names that are hardcoded in its executable or that reside in memory and can be used to expose it. Thus, simply modifying these strings, functions, and module names can often lead to lower rates of detection, at least by more traditional anti-malware solutions. This is one of the most basic ways to abuse signature-based anti-malware detections, but modern malware requires more sophisticated evasion techniques to deal with more sophisticated anti-malware solutions. This is where mutation comes in.

Mutation is malware's ability to change its features and code in order to evade detection or adapt to its environment. With mutation, the malware's goal is to change its code just enough to fly under the radar while retaining its basic functionalities. To help you better understand mutation, let's look at the life cycle of a typical malware sample.

Some malware authors, notably cybercrime groups, "spam" malware to hundreds or thousands of potential victims, expecting to trick a few unsuspecting people into executing it. Once enough victims have been infected by a strain of malware, investigators will inevitably get their hands on the sample and develop detection rules for it, which will then be im-

plemented in anti-malware engines and other defense technologies. This can quickly reduce the lifespan of any given piece of malware. This doesn't apply just to mass spam malware, however; it can also be true of more targeted malware. If the malware is able to mutate on the victim host, it may be able to similarly evade detection.

There are different forms of mutation, such as *code block reordering*, in which code is shifted around and reordered to create new "variants" of the malware, and *register reassignment*, in which CPU registers are changed (for example, all ECX register references are changed to EDX). These mutations of the malware's code, implemented by a *mutation engine*, can significantly change the code's structure and signature. The following is a very simple example of a mutation that shows how assembly code may be functionally identical but appears different:

```
mov eax, 0
add eax, 1
```

This example code block simply sets the `eax` register to 0 and then sets `eax` to the value of 1. Compare this to the following code:

```
xor eax, eax
inc eax
```

This code also sets `eax` to 0 (but using the `xor` instruction rather than the `mov` instruction) and then sets `eax` to 1 (using the `inc` instruction instead of `add`). If this were actual malicious code, anti-malware software might have a detection signature for one of these code blocks, but perhaps not the other. In the real world, of course, this code would be a lot more complex.

Mutation can occur either dynamically during runtime or on the malware file itself. Mutation during runtime occurs once the malware executes on the victim system. The malware may dynamically alter its code in memory to evade defenses that scan for malicious code patterns in memory. Mutation on the malware file itself occurs before the malware is delivered to the victim. The malware author may run their code through a mutation engine that spits out unique variants of the same malware that can all be delivered to different victims. Packers, which I'll discuss in detail in [Chapter 17](#), can be a form of both runtime and static file mutation. When malware is run through a packing engine, it is obfuscated so that it will appear unique to all other variants of the same malware family.

The virus-like ransomware malware family Virlock provides a good example of mutation. When Virlock (SHA256:

7a92e23a6842cb51c9959892b83aa3be633d56ff50994e251b4fe82be1f2354c) executes on a victim system, it decrypts three instances of itself in memory and drops them to the disk as files. All three instances differ from all other Virlock malware samples and have a different signature. This ensures the malware will remain undetected, at least by anti-malware engines that rely on hashes and basic file signatures for detection.

Using Uncommon Programming Languages

In an attempt to circumvent endpoint defenses, malware authors may use obscure or uncommon programming languages to develop their malware. Anti-malware software may be unfamiliar with the code and data structure in these languages, and it takes time for its signature and heuristic detections to catch up. Uncommon or new programming languages can also create a challenge for malware analysts and reverse engineers who are expecting more typical malware code, such as C or C++. Furthermore, many of these uncommon languages can be used across different operating systems. For instance, a program written for Windows can also run on macOS or Linux as long as the victim system has the required libraries installed. This can make the malware more resilient to different operating systems.

Using uncommon languages isn't a new technique. Early malware was often written in C, but malware authors began using the .NET framework (such as C#), which is still very popular. However, anti-malware and other defenses have caught up, so malware authors are adapting and now increasingly using other languages. Python, a very common scripting language, has seen more use in malware in both scripts and executable format. Malware authors simply code a malicious Python script (.py), which can be executed on any system that has the right Python libraries installed. Tools such as Py2Exe and PyInstaller can even convert a Python script into an executable, which the attacker can deploy on a victim machine similar to a standard PE file. Since it's fairly trivial to code a malicious Python script and then convert it into an executable, this approach has a lower barrier to entry for malware authors.

Nim (<https://nim-lang.org>), a self-proclaimed "statically typed compiled systems programming language," has also seen growing use with malware authors. Notably, Nim was used by the threat group behind the infamous Trickbot banking trojan (see Lawrence Abrams's article "TrickBot's BazarBackdoor Malware is Now Coded in Nim to Evade Antivirus" at <https://www.bleepingcomputer.com/news/security/trickbots->

[***bazarbackdoor-malware-is-now-coded-in-nim-to-evade-antivirus/***](#)).

Using Nim potentially helped the malware evade AV defenses.

Go (sometimes called Golang) is an open source language from Google. It's simple to program in (compared with other compiled languages such as C), so it's no surprise that it has also seen increased use in malware. Rust ([***https://www.rust-lang.org***](https://www.rust-lang.org)) is another language seeing more use in malware development. Indeed, nearly any programming or scripting language can be used for nefarious purposes, so this section could get very long. Even PowerShell is being used more often by malware authors. Malware analysts and defenders would be wise to stay abreast of the different languages being used in modern malware.

Abusing Certificate Trust and Signing

A *digital signature* is a trusted certificate that functions as a mark of approval for a file, informing the operating system and other applications that the file is legitimate and safe to execute. Many anti-malware solutions put less scrutiny on files that are digitally signed by a known and trusted authority. As a result, malware can abuse certificate trust chains to evade endpoint defenses.

The *certificate trust store* is the repository where Windows stores the signer certificates that it trusts. You can view the trust store on Windows with the certmgr application (which you can find in `C:\Windows\System32`), as shown in [***Figure 13-12***](#).

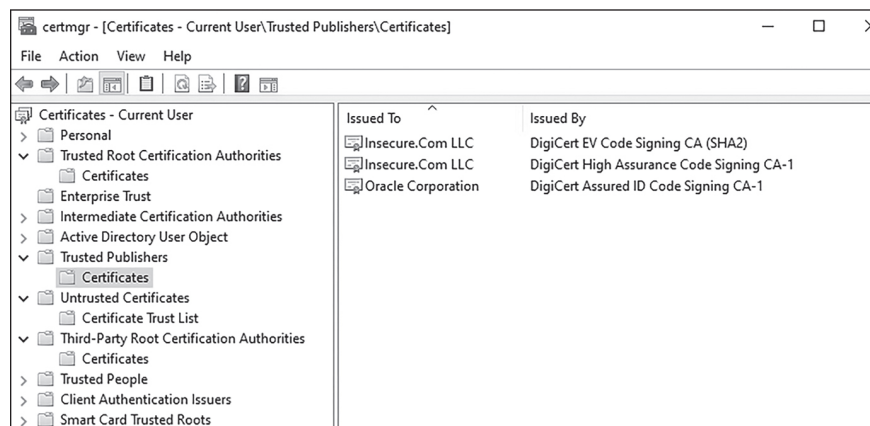


Figure 13-12: Viewing the trust store in Windows with certmgr

Malware authors can digitally sign their code with a trusted certificate in a few ways. First, if a threat actor were to infiltrate the network of a trusted company, it could generate valid certificates and sign its own malware with them, or it could steal certificates and digitally sign its malicious code with those. In fact, this has happened on more than one occasion, such as when code-signing certificates were stolen from Nvidia in

2022 (see Pieter Arntz's article "Stolen Nvidia Certificates Used to Sign Malware—Here's What to Do" at <https://www.malwarebytes.com/blog/news/2022/03/stolen-nvidia-certificates-used-to-sign-malware-heres-what-to-do>). Code-signing certificates are sometimes even sold on the dark web!

Second, in certain circumstances it's possible to insert malicious code into a previously signed executable file, all without invalidating the certificate. This was presented by researchers at the Black Hat conference in 2016 (see the report "Certificate Bypass: Hiding and Executing Malware from a Digitally Signed Executable," from the Deep Instinct Research Team, at <https://www.blackhat.com/us-16/briefings/schedule/#certificate-bypass-hiding-and-executing-malware-from-a-digitally-signed-executable-3984>).

Finally, malware can simply add a certificate to the list of trusted certificates on the operating system. This is easier said than done: the malware must already be executing on the host with a high level of privileges. But if it succeeds, the malware author may be able to run any additional malware they wish to on the host.

Abusing Engine Limitations

As mentioned earlier, anti-malware software (as well as EDR software that has built-in anti-malware capabilities) has the difficult task of balancing high rates of detection with the end-user experience. This means, first and foremost, that it must have a high success rate when detecting malicious code, while also limiting false positives. Additionally, the anti-malware scanning and monitoring activity must be transparent to the end user. These scans shouldn't impact the user experience at a level where the system becomes unusable or unstable. Cunning malware authors can take advantage of these restrictions using various techniques, two examples being delayed execution and memory bombing.

In the same way that some malware uses delayed execution to evade detection inside a sandbox (as discussed in [Chapter 8](#)), it may be able to "outsleep" an anti-malware scanning engine. One way it can do this is by waiting until a specific number of milliseconds have passed. Once, say, 600,000 milliseconds (10 minutes) have elapsed, the sample executes its malicious code. Because the anti-malware engine might have a time limit on its emulation or sandboxing engine (to prevent the anti-malware engine from indefinitely analyzing a large file and consuming valuable system resources), this technique sometimes allows malware to slip through undetected.

Memory bombing, a term coined in the book *Antivirus Bypass Techniques* by Uriel Kosayev and Nir Yehoshua (Packt, 2021), involves malware allocating excessively large regions of memory inside its process address space. Because anti-malware software must take into account its system resources consumption (CPU, memory, and so on), it might only quickly scan this large region of memory or even ignore it altogether, allowing the malicious code to go unnoticed. Note that this technique can also be effective against sandboxes.

Masquerading as a Safe File

Masquerading is used by malware authors to disguise malware as legitimate files. This technique is used primarily as a method of deceiving a victim rather than as a means of directly circumventing endpoint defenses. Masquerading can take many forms, including the following:

Spoofing a filename

The malware author simply names a malicious file after a common system file or legitimate application file (such as *explorer.exe* or *PowerPoint.exe*) or slightly modifies a filename (such as *expl0rer.exe*). Malware authors could also change the file extension of malicious files to something more unassuming, such as renaming an *.exe* file to a *.jpg* file.

Spoofing file metadata

The malware author spoofs the metadata of a malicious file, such as by using “Microsoft” as the file’s publisher or company name. A similar technique is reusing legitimate program icons. For example, malware might use the Microsoft Word logo to make its malicious file appear genuine.

Executing social engineering attacks

The malware author tricks the user into executing the malware, for example, by sending an email to a target user with a malicious file called *important_invoice.pdf*. A similar technique is the use of double extensions. By default, Windows doesn’t display file extensions, so the file *financials.xls.exe* would display in Windows as simply *financials.xls*. This may mislead an unsuspecting person into launching a malicious executable file.

Even though masquerading is a relatively simple and inexpensive technique to implement, it can be quite effective. Notably, researchers from ESET reported that the threat group Sandworm delivered to victims in Ukraine malware disguised as a component of the IDA Pro disassembly tool and ESET’s own security software (see Kelly Jackson Higgins’s article

“Sandworm APT Trolls Researchers on its Trail as it Targets Ukraine” at <https://www.darkreading.com/threat-intelligence/sandworm-apt-trolls-researchers-on-its-trail-while-it-targets-ukraine>). Another notable example of masquerading is malware imitating documents authored by reputable organizations, such as COVID-19–related information from the World Health Organization, as reported by Proofpoint in 2022 (see “Nerbian RAT Using COVID-19 Themes Features Sophisticated Evasion Techniques” from Andrew Northern and colleagues at <https://www.proofpoint.com/us/blog/threat-insight/nerbian-rat-using-covid-19-themes-features-sophisticated-evasion-techniques>).

So far, we’ve explored different techniques malware uses to circumvent host defenses by actively tampering with them or passively slipping by them. Now let’s shift gears a bit and explore privilege elevation and how it can be used by malware to carry out operations that may otherwise be detected and blocked by endpoint defenses.

Privilege Elevation for Defense Evasion

Privilege elevation, or obtaining a higher level of privileges than one currently holds, can be a potent evasion tactic. After obtaining a high privilege level on the host, an attacker has a lot more freedom to execute further attacks that bypass endpoint defenses. As you’ve seen, high privilege levels are required for actions such as disabling anti-malware or modifying firewall configurations. While there are many ways to elevate privileges, this section will focus on four of the most common techniques in modern malware: UAC bypasses, access token impersonation and manipulation, credential reuse, and direct exploitation.

Bypassing User Account Control

User Account Control (UAC) is a protection control in Windows designed to prevent unauthorized applications from executing code at high privilege levels. When an application requests administrator access, an administrator on the system must consent to the request (see [Figure 13-13](#)).



Figure 13-13: A typical UAC pop-up

When an administrator consents, the application’s process integrity level increases to “high.” The most common integrity levels (high, medium, low, and system) help dictate what a process can and cannot do on a system. High-integrity processes run in an elevated mode and have access to objects assigned to lower integrity levels. Medium-integrity processes run with standard user rights; this is the default setting for most processes. Low-integrity processes have the lowest privilege level and are usually reserved for applications such as web browsers that should run in a container-like environment for security reasons. System integrity processes are those that are integral to the stability of the operating system; these include the Service Control Manager (*services.exe*) and the Windows subsystem process (*csrss.exe*). By design, processes with lower integrity levels cannot modify data in processes with higher integrity levels.

Figure 13-14 shows an excerpt from Process Hacker, which conveniently highlights processes based on their elevation and integrity levels.

explorer.exe	4316	0.21	31.11 MB	DESKTOP	Windows Explorer
ProcessHacker.exe	6112	2.71	16.92 MB	DESKTOP	Process Hacker
notepad.exe	4824		2.85 MB	DESKTOP	Notepad
WINWORD.EXE	7160		24.98 MB	DESKTOP	Microsoft Word
EXCEL.EXE	4880		16.66 MB	DESKTOP	Microsoft Excel
cmd.exe	6528		4.02 MB	DESKTOP	Windows Command Processor
conhost.exe	3044		2.64 MB	DESKTOP	Console Window Host

Figure 13-14: Process integrity viewed in Process Hacker

Process Hacker itself (*ProcessHacker.exe*) is highlighted in orange (or dark gray in this book), meaning that it’s elevated and running in a high-integrity mode. *Explorer.exe* is highlighted in pink (medium gray), demarcating it as a system process. The other, more mundane processes, such as Excel and Notepad, are yellow (light gray), meaning that they’re medium-integrity processes running with standard user privileges.

UAC works to protect systems against malicious privilege elevation attempts by explicitly requesting permission from a higher-level account. UAC bypass attacks rely on tricking a user, an application, or the operating system itself into executing potentially dangerous actions in an elevated context. Let's take a look at how UAC bypasses work in practice with a simple example.

A number of built-in Windows utilities are designed to be run with elevated permissions. One of these utilities is *msconfig.exe*, a simple Windows configuration tool that allows system administrators to change Windows boot options and modify services and startup tasks, among other things. Normally, applications requesting elevated permissions produce a UAC prompt; by default, even users with administrator privileges must consent to this prompt. If executed from an administrator account, however, *msconfig.exe* automatically elevates itself to a high-integrity process without prompting for UAC. Furthermore, it allows for the execution of other tools that will subsequently run in a high-integrity context, also without a UAC prompt. Unfortunately for *msconfig.exe*, an actor can abuse this behavior, resulting in a simple UAC bypass. From the Tools menu in *msconfig.exe*, a user can select the Command Prompt tool and click Launch (see [Figure 13-15](#)).

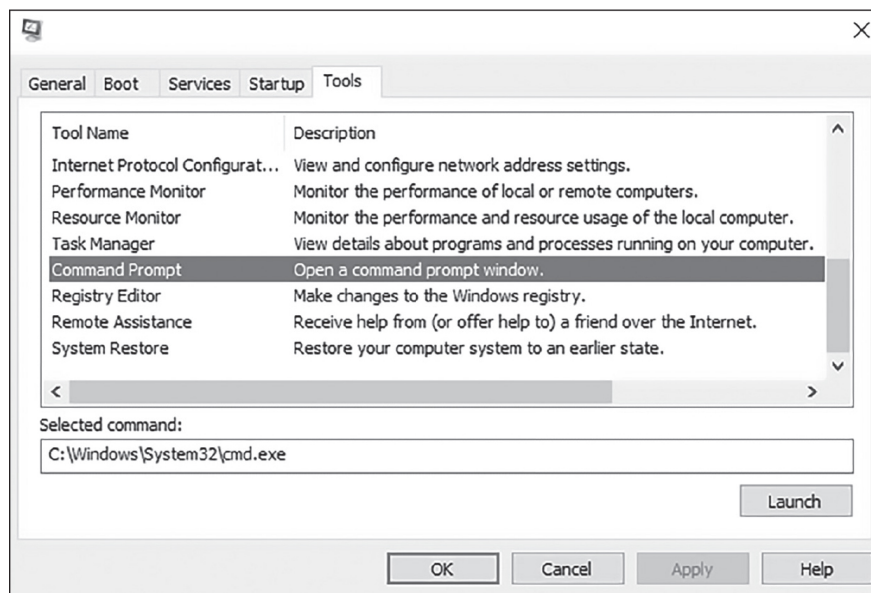


Figure 13-15: Launching a command prompt from *msconfig.exe*

Next, a new command prompt with the integrity level of its parent process (*msconfig.exe*) will launch, without prompting for UAC permission. [Figure 13-16](#) shows this new *cmd.exe* process in Process Hacker.

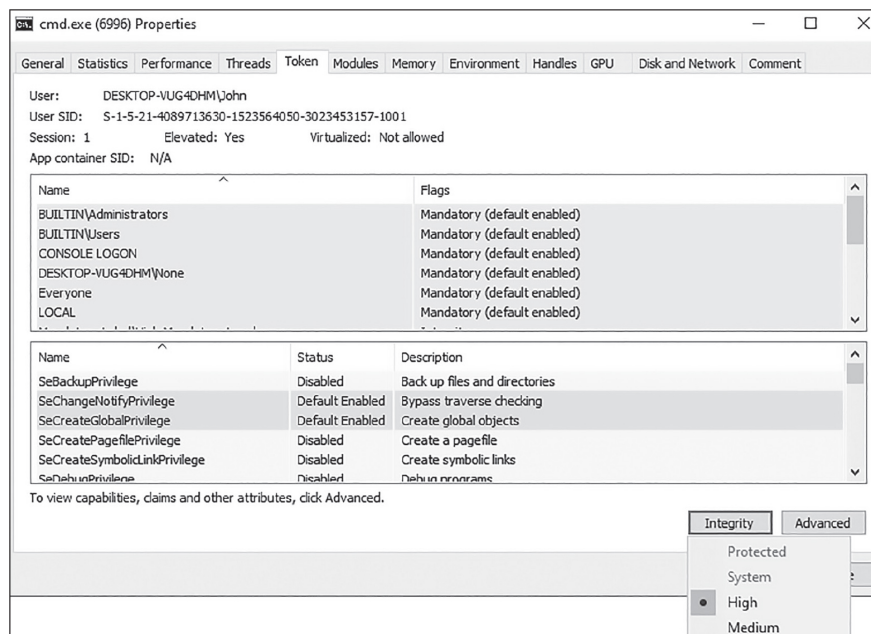


Figure 13-16: The `cmd.exe` process spawned with high integrity

Notice that the process's privileges are elevated ("Elevated: Yes") and its integrity is high. The user bypassed UAC by executing a process that must be run in a high-integrity context (which doesn't prompt for UAC permission on the default Windows configuration) and by spawning a command prompt that inherits that process's high integrity level. This command prompt is now running as administrator and can be used to execute high-privilege commands. A similar real-world example of this comes from the malware family Trickbot, which abuses the auto-elevation property of the `WSReset.exe` Windows tool; see the article "Trickbot Trojan Leveraging a New Windows 10 UAC Bypass," by Arnold Osipov at <https://blog.morphisec.com/trickbot-uses-a-new-windows-10-uac-bypass>.

The problem with this particular UAC bypass is that it requires GUI access to the system, so malware would have to jump through quite a few hoops to execute this attack without the victim noticing what's going on. Malware authors have discovered a few ways to get around this limitation, however.

DLL Hijacking

In [Chapter 12](#), you learned that malware can abuse hijacking vulnerabilities in legitimate applications to inject a malicious DLL and stealthily run code. This type of attack is also an effective way to bypass UAC. Revisiting the `msconfig.exe` example used earlier, if it fell prey to DLL hijacking and allowed a malicious DLL to be loaded in place of a legitimate one, the malicious DLL could then execute under the context of the high-privileged `msconfig.exe` application. Any UAC pop-ups will be under the guise of `msconfig.exe`, possibly tricking the user into consenting and allowing the

malicious code to run at a higher privilege level than would otherwise be possible.

In a 2016 report from Fox-IT and NCC Group called “Mofang: A Politically Motivated Information Stealing Adversary” (which you can download at <https://blog.fox-it.com/2016/06/15/mofang-a-politically-motivated-information-stealing-adversary/>), researchers highlighted such an attack by the threat group Mofang, which abused the legitimate *migwiz.exe* Windows application to load a DLL called *cryptbase.dll*. This DLL was hijacked by the attackers, causing *migwiz.exe* (which runs in an elevated state by default) to load the malicious version of *cryptbase.dll*, effectively bypassing certain UAC controls.

COM Abuse

The *Component Object Model (COM)* is a part of the Windows API that allows for interprocess communication. The basic building block of COM is a *COM object*, which is composed of data and the functions that control access to it, known as *interfaces*. A COM object server exposes interfaces to COM clients, and clients access the COM server via these interfaces. COM server objects are often *.dll* or *.exe* files. Each COM server object has a unique ID, called a class ID (CLSID), which is a 128-bit string that takes the form of a series of numbers and characters. These strings are often displayed in brackets, as in this example: {4E5FC2F8-8C44-6776-0912-CB15617EBC13}. This will be important in a moment.

A number of COM objects have a property called *COMAutoApproval*, which indicates that the particular COM object doesn’t require a user to explicitly permit the privilege elevation functions that are part of the object. On a Windows system, you can view the list of COM objects that have this property in the registry key *HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\UAC\COMAutoApprovalList*. **Figure 13-17** shows this list on my system.

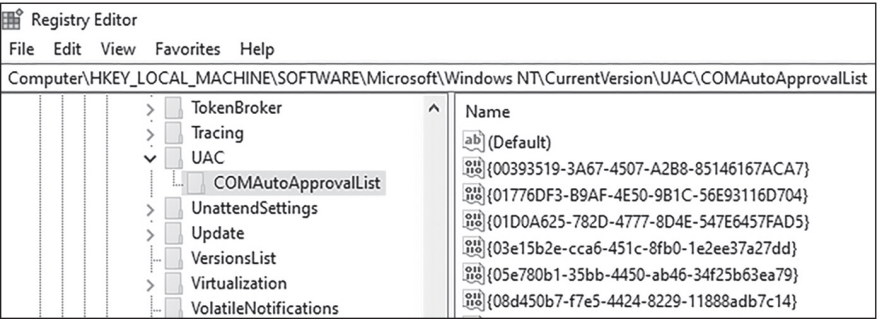


Figure 13-17: Viewing COM objects with COMAutoApproval enabled

On a typical Windows 10 system, there are over 100 objects in this list! As you might imagine, some of these objects can be abused by malware to bypass UAC. For example, some COM object interfaces contain functions that can execute arbitrary code. A COM interface may expose a function called `ExecuteCommand`, for instance, that takes an arbitrary command as input. The following pseudocode illustrates this:

```
command = "cmd.exe copy malware.exe C:\System32\malware.exe"
CoInitialize(NULL);
comObj = CoCreateInstance (CLSID_VulnerableComObject, ..., ...,
IID_VulnerableComObject, &pMethod);
pMethod->ExecuteCommand(command);
```

This malware initializes COM (`CoInitialize`) and creates a COM instance (`CoCreateInstance`) of the COM object `VulnerableComObject`. Once the COM object is initialized, the malware invokes the vulnerable `ExecuteCommand` method (which is exposed and exported via the COM object), passing `command` as a parameter. Because the COM object is running with high privileges, the subsequent command will also be executed with high privileges. This specific command copies the malware executable into the `System32` directory, an action that would normally display a UAC prompt.

This is a simplified version of this technique, but similar techniques have been used in real-world attacks. A well-researched project called UACME details a variation of this technique and many more at <https://github.com/hfiref0x/UACME>.

Registry Tampering

Although modifying the registry often requires high-level privileges on the victim system in the first place, malware can use the registry to side-step the UAC prompt itself. Consider a scenario in which an attacker has control over the infected host and can execute commands on it but doesn't have a GUI to the host. The attacker might still be required to click on UAC prompts to execute certain actions, and this can be problematic without a graphical interface to the victim system.

In this case, the malware can try to modify the registry key `HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Policies\System\EnableLUA` from the default value of 1 to 0, which will effectively disable UAC prompts on the host. Setting the key `HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Policies\System\ConsentPromptBehavior` to 0 will also disable UAC prompts for administrator actions.

Note that the machine may need to be rebooted in order for these changes to take effect, which could be problematic for the attacker and provoke unwanted attention.

Impersonating and Manipulating Access Tokens

Each process running on a system has an assigned access token inherited from its parent process, whose access token is in turn inherited from the user account that spawned it. This user account access token represents the rights and privileges of the user account. Process access tokens can be assigned different privileges: `SeBackupPrivilege`, for example, grants a user or process the ability to back up files and directories; `SeLoadDriverPrivilege` grants the process the ability to load a kernel driver; and `SeShutdownPrivilege` permits a process to shut down the system.

To gain additional privileges, malware can duplicate a token from another process, a technique often known as *token theft* or *token impersonation*. The malware calls `DuplicateToken` or `DuplicateTokenEx` to create a copy of a token assigned to another process, then calls `ImpersonateLoggedOnUser`, passing the duplicated token as a parameter. This assigns the permissions of the original token to the malware. The malware can also call `CreateProcessWithToken` to spawn a brand-new process and assign it the permissions of the duplicated token.

NOTE

Alternatively, malware can manipulate its own token and assign it new privileges. One such coveted privilege that malware may attempt to obtain is `SeDebugPrivilege`, which permits a process to inspect and manipulate other processes owned by other user accounts, up to and including the System account. If a lower-privileged process is allowed to manipulate System processes, it can potentially inject and execute code in the context of these processes, completely bypassing UAC controls or, at the very least, tricking users into permitting elevated actions via UAC.

To gain the `SeDebugPrivilege` privilege, the malware calls the `OpenProcessToken` function to open the access token of its process, then invokes `AdjustTokenPrivileges`, passing `SeDebugPrivilege` as a parameter. However, just because a process requests this privilege doesn't mean the operating system will grant it. Only previously elevated processes can request and be granted this permission, so the malware must already be running in an elevated state. Malware usually will use one of the UAC bypass techniques mentioned earlier in this chapter to elevate its privileges

and then grant itself `SeDebugPrivilege` to gain further access to other system processes.

Extracting and Reusing Credentials

Locating and extracting account credentials can be a potent privilege elevation method. If malware can locate an administrator's credentials, for example, it may be able to reuse them to create a process with elevated permissions, or even to move laterally to other hosts on the network and gain elevated privileges on those systems. There are many ways malware can do this.

One method of extracting credentials is to inspect the memory of the *Local Security Authority Subsystem Service (LSASS)* process, which oversees security policies and authentication. By nature, its process (*lsass.exe*) contains sensitive data in its memory address space—namely, login credentials and security tokens. On modern versions of Windows, this sensitive data is obfuscated (for example, via hashing), but in special circumstances it may be in cleartext. By inspecting the process memory of *lsass.exe*, malware may be able to locate and extract privileged credentials and tokens, then attempt to reuse them to elevate its own privileges.

There are some tools that can automate this process. One well-known example, Mimikatz (<https://github.com/gentilkiwi/mimikatz>), can extract credentials (and other sensitive data) from the victim system's memory. Even though Mimikatz has existed since 2011, it is still actively updated and often used in real-world attacks. Some attack frameworks, such as Cobalt Strike, even have tools like Mimikatz built in.

Another method of credential extraction is *keylogging*, an effort to capture the victim's keystrokes to obtain login credentials for various accounts and services. Keylogging is often implemented with different forms of hooking, as described in [Chapter 12](#).

Exploiting Vulnerabilities for Privilege Elevation

Finally, some malware may attempt to exploit vulnerabilities in order to elevate its privileges. *Local privilege elevation (LPE)* vulnerabilities, which allow a malware to elevate its privileges on a system it already has lower-level access to, are especially popular among threat actors. One notable, relatively recent example of exploitation for privilege escalation is the CVE-2021-36934 vulnerability (see <https://cve.mitre.org>), nicknamed "HiveNightmare," which affected Windows 10 and some versions of Windows 11. HiveNightmare takes advantage of an access control issue in certain Windows directories. Inside these directories lie registry hives

containing sensitive data, such as stored credentials, that’s not normally accessible to underprivileged users. However, by exploiting this vulnerability, an attacker could access backups of this sensitive data inside the Windows *Volume Shadow Copy Service (VSS)*, a Windows backup mechanism that stores copies of files and other data. In vulnerable versions of Windows, the attacker could craft a special payload that exploits this access control flaw, allowing the attacker to access unlocked copies of the sensitive registry hive data and use the stored credentials to elevate their privileges on the system.

Figure 13-18 illustrates this exploit in action. (The exploit code can be obtained at <https://github.com/GossiTheDog/HiveNightmare>.)

```
C:\Users\Public>HiveNightmare.exe
HiveNightmare v0.6 - dump registry hives as non-admin users
Specify maximum number of shadows to inspect with parameter if wanted, default is 15.
Running...
Newer file found: \\?\GLOBALROOT\Device\HarddiskVolumeShadowCopy1\Windows\System32\config\SAM
Success: SAM hive from 2022-01-21 written out to current working directory as SAM-2022-01-21
Newer file found: \\?\GLOBALROOT\Device\HarddiskVolumeShadowCopy1\Windows\System32\config\SECURITY
Success: SECURITY hive from 2022-01-21 written out to current working directory as SECURITY-2022-01-21
Newer file found: \\?\GLOBALROOT\Device\HarddiskVolumeShadowCopy1\Windows\System32\config\SYSTEM
Success: SYSTEM hive from 2022-01-21 written out to current working directory as SYSTEM-2022-01-21
Assuming no errors above, you should be able to find hive dump files in current working directory.
```

Figure 13-18: The *HiveNightmare* exploit in action

This exploit code dumps the extracted registry hive files into the current working directory. A quick directory listing reveals that the exploit was successful (see **Figure 13-19**).

10/16/2022	08:33 PM	65,536	SAM-2022-01-21
10/16/2022	08:33 PM	65,536	SECURITY-2022-01-21
10/16/2022	08:33 PM	11,796,480	SYSTEM-2022-01-21

Figure 13-19: The *HiveNightmare* exploitation was successful!

This attack may not look like much from the captured screenshots, but it’s powerful. As a nonprivileged user, I was able to extract sensitive registry hives directly from VSS that otherwise would not be available to me; had I wanted to, I could have queried them for stored credentials that I could then have used to execute code at a high privilege level. This is just one example of a vulnerability (and also a good example of credential extraction), but numerous privilege elevation vulnerabilities have been released over the past five years; some are high profile, and some are actively being abused by malware in the wild. Unfortunately, there are likely more similar vulnerabilities that are still unknown to the public.

Now that we've looked at some ways in which malware might circumvent endpoint defenses, let's shift our focus to how it could circumvent network defense controls.

Circumventing Network Defenses

Network controls are another layer of defense that malware must bypass in order to be successful. Circumventing network defenses relies on many of the tactics you've already seen in previous chapters. For example, malware can take advantage of obfuscation techniques such as encryption to conceal its C2 traffic. It can also leverage sandbox evasion techniques to circumvent network defenses that rely on sandboxing of suspicious files traversing the network. In this section, I'll focus on circumvention techniques that haven't yet been covered in the book. Before we start, however, I'll briefly introduce some of the various network defenses available.

Introducing Modern Network Defenses

As with host defenses, the network defense market is flooded with product names and acronyms. At a fundamental level, there are only a few different types of modern network defenses. The ones I'll focus on in this chapter are NIDS, NDR, and email protection:

Network intrusion detection systems (NIDS)

These products monitor incoming and outgoing network traffic, searching for signs of malicious activity. Much like anti-malware, NIDS can be signature based, anomaly based, or a combination of the two. Signature-based NIDS techniques search for suspicious signature patterns in network traffic such as sequences of data, code, or strings. Anomaly-based techniques look more into behavioral patterns of network traffic and may leverage machine learning techniques. NIDS can also be *intrusion prevention systems (IPS)*, which not only detect attacks but also respond to and prevent them. Both NIDS and IPS are often included in modern implementations of more traditional network defenses like firewalls.

Network detection and response (NDR)

This is the network equivalent of EDR. NDR is a more modern solution than NIDS and in some ways seeks to replace it. NDR uses real-time network traffic analysis to identify potential attacks, detect threats (such as malware) traversing the network, and be triggered by suspicious behaviors. The telemetry of NDR and host-based EDR can be combined to create extended detection and response (XDR), which allows analysts and investigators to track attacks across both the network and endpoint layers.

Email protection technologies

These solutions, which are becoming more widely used and necessary, sit at the email gateway of an organization and monitor and inspect inbound and outbound email traffic. This is a crucial control, as many inbound emails are laced with malicious attachments or URLs. Email protection suites scan and sandbox inbound and (sometimes) outbound emails, then raise alerts on (or simply delete) malicious ones. Since this technology partly relies on sandboxing, it can potentially be thwarted by the previously discussed anti-sandbox techniques.

Now we'll start looking at the ways in which malware might try to circumvent these controls.

Obfuscating and Obscuring Network Traffic

To bypass NIDS and NDR, modern malware must employ obfuscation or encryption techniques to mask its network traffic. Some malware may attempt to hide from network defenses while downloading additional payloads or modules using encrypted network protocols such as HTTPS, Secure File Transfer Protocol (SFTP), or Secure Shell (SSH). Malware has even been known to make use of the encrypted surveillance protection software Tor to obscure its network activities. The malware family Bashlite (also known as Gafgyt) has been observed using this technique when communicating with its C2 infrastructure. Keep in mind that the very act of using Tor and other lesser-used protocols and services can itself be a telltale sign of malicious activity. While this technique may indeed prevent network defenses and investigators from inspecting the suspicious traffic, it doesn't mean the malware will go completely undetected.

Another example of this tactic is *DNS tunneling*, which abuses the Domain Name System (DNS) protocol to hide traffic such as file downloads, exfiltration of data from the network, or C2 communication. Since DNS is a fundamental protocol that the internet runs on, DNS tunneling may go completely unnoticed by network monitoring and defenses. [Figure 13-20](#) illustrates what DNS tunneling looks like.

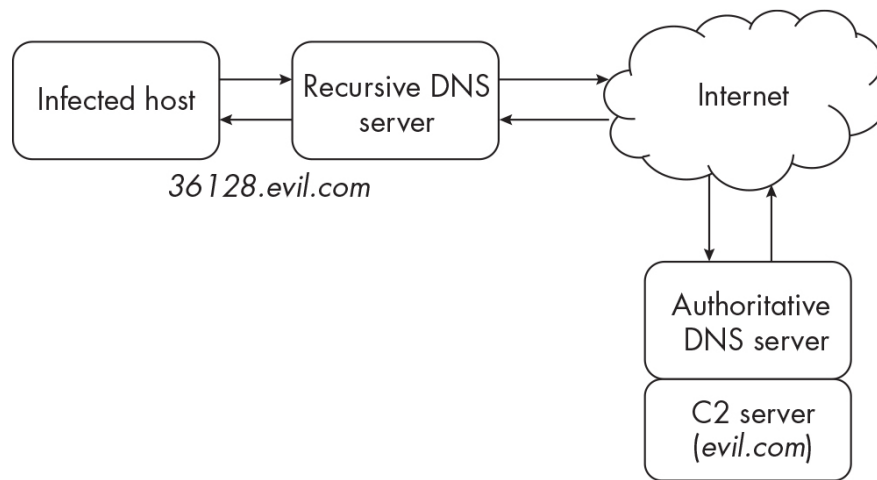


Figure 13-20: DNS tunneling

Let's start with the infected host, which has a remote access trojan (RAT) hiding on it. This RAT communicates with its C2 server (*evil.com*) and requires special instructions that it will receive via DNS tunneling. It sends a DNS query to its C2 server (*36128.evil.com*). The *36128* subdomain is a randomized numerical string. Next, the DNS request is sent to a recursive DNS server, a local server that services DNS requests. This recursive DNS server may be an internet service provider (for home users and small businesses) or a server inside an organization that the host is part of.

Recursive DNS servers subsequently contact an *authoritative DNS server*, the source of authority for a domain. When a new website is provisioned from a hosting provider, the provider acts as the authoritative DNS server for that domain. Alternatively, anyone can establish their own authoritative DNS server for a domain name they have purchased. In this case, the RAT on the infected host is ultimately contacting the authoritative DNS server for the domain *evil.com*, and the malware author owns the authoritative DNS server for this domain.

Once the C2 server receives the DNS request, it responds with a reply to the sender. In this case, however, the reply includes an encoded command, the simple string *execute*. This command, which could be obfuscated with a simple algorithm such as Base64 or even encrypted, can be hidden away in one of the records inside a DNS response. Record types that are often abused for DNS tunneling are TXT, CNAME, Resource Records (RR), and NULL records. (These are outside the scope of this book and won't be discussed further.)

Once the infected host receives the DNS response, the RAT decodes the embedded string and processes the command *execute*. Presumably, this means the RAT will then execute a malicious command. DNS by default is not encrypted, but again, since it's so widely used and may be difficult to fully monitor, this type of attack may slip past network defenses.

Malware has also been known to use custom C2 frameworks that employ novel methods of network communication. These frameworks may leverage publicly accessible services such as GitHub, Pastebin, Telegram, or even X to conceal traffic and commands. Not only is this traffic usually encrypted, but it also can go completely unnoticed since it appears normal. One example of this arose when researchers at ESET discovered that the threat group Turla abused Britney Spears's Instagram account to conceal its C2 servers (see the article "Carbon Paper: Peering into Turla's Second Stage Backdoor" at <https://www.welivesecurity.com/2017/03/30/carbon-paper-peering-turlas-second-stage-backdoor/>). The Turla group pasted to Spears's Instagram page encoded text representing a URL that contained the address of its C2 server. Malware on systems previously infected by the Turla group then monitored her Instagram profile for these specific patterns to retrieve the address information of the C2 servers. The address was then decoded by the malware and used to retrieve additional commands from the attacker's infrastructure.

Attackers may also host their malware payloads on well-known file-sharing sites like Google Drive and Dropbox. Since these services are widely used for legitimate purposes, malware that's downloading a payload or additional modules from Google Drive, for example, may look completely normal to a casual observer and to network defenses.

Concealing Infrastructure Using Geofencing

Geofencing refers to the use of geographical locations as a factor that determines the malware's behavior. In essence, the malware is designed to execute its payload only when the target host is located within a specific geographical boundary, such as a specific country. Geofencing can also be used to deter and prevent malware-scanning engines and analysts from identifying and investigating a malicious server. In the case of email protection solutions, for example, as malicious attachments are detonated in a sandbox, the sandbox may identify that the malware is attempting to communicate with an unknown domain on the internet. The sandbox may then try to "scan" or probe the server in an effort to identify its true nature and classify it as either legitimate or malicious.

Geofencing techniques can be used by a malicious server to prevent this behavior. By querying the location of the system or investigator probing its infrastructure, the server can hide itself from uninvited guests. A malware author in China, for example, may be specifically targeting victims in Germany. Any traffic to the malware's C2 server not originating from an IP address in Germany can be blocked, preventing automated scanning engines or investigators without a German IP address from inspecting it more closely. Alternatively, the server could even display mislead-

ing information to systems not originating from Germany. For instance, any traffic from non-German IPs may be rerouted to a completely different, benign website. A similar technique was identified by researchers at Proofpoint, who revealed that the threat group TA547 leveraged geofencing to serve malware payloads to only certain targets (see the article “The First Step: Initial Access Leads to Ransomware” at <https://www.proofpoint.com/us/blog/threat-insight/first-step-initial-access-leads-ransomware>). Payloads were hosted on servers, and only victims originating from specific countries were allowed to access and download the malware payloads.

Generating New Infrastructure Using DGAs

One major problem that malware authors face is the fact that once a C2 server is identified by a sandbox, investigator, or defense software, it is effectively “burned,” meaning it will soon be blocklisted by most security products. This renders the malware that communicates with this server ineffective.

Malware can get around this problem by dynamically generating new C2 server addresses using *domain generation algorithms (DGAs)*, which consist of client- and server-side components. The malware (the client, in this case) uses embedded DGA code to create new domain names. The malware’s C2 server uses the same algorithm to produce a new domain name identical to the one created by the malware client. DGA operates on the principle that both the client and server sides of the algorithm generate predictable domain names. This algorithm must generate the same domain name on both the client and server sides, but it must be unpredictable enough that security researchers and analysis tools can’t guess the next domain names that will be generated.

Figure 13-21 illustrates how DGAs are used by malware.

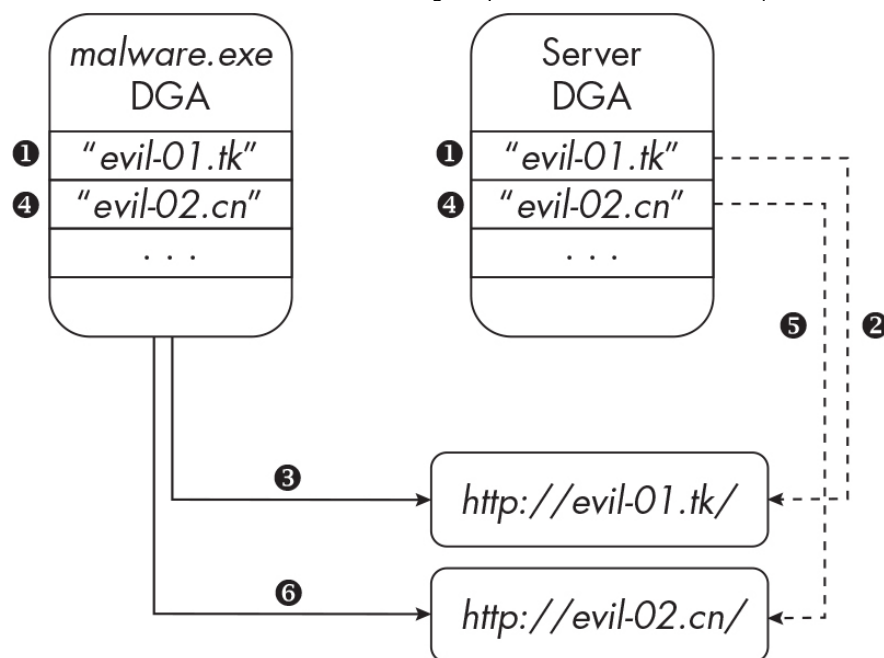


Figure 13-21: How DGAs work

The malware first generates a new domain name using DGA code embedded in its executable file. After executing the algorithm, the malware produces the domain *evil-01.tk*, and on the malware's C2 infrastructure, the same algorithm runs and also produces *evil-01.tk* ❶. Then the malware author provisions a new server with the address *evil-01.tk* ❷. The malware then connects to the C2 server using this domain name, which is now online ❸.

After a predetermined period of time, such as eight hours, the malware generates a new domain, *evil-02.cn* ❹. In turn, the server generates the same domain name, which is then provisioned ❺. Finally, the malware connects to it ❻. By this time, the first server (*evil-01.tk*) is likely offline. This cycle repeats until the algorithm runs out of available domains. This powerful DGA technique allows malware authors to generate hundreds or thousands of new domain names to use as C2 servers, making it a game of “whack-a-mole” for security products and investigators to identify and add these domains to a blocklist.

Executing the Fast-Flux Technique

Fast flux is a type of evasion technique used in large part by *botnets*, networks of infected systems that an attacker controls. As victims are infected by a particular malware variant, the infected hosts (the *bots*) are added to the botnet. Botnets are used by threat actors for various purposes, such as to send spam, conduct phishing and *distributed denial-of-service (DDoS)* attacks, and facilitate various types of fraud. The fast-flux

technique allows threat actors to use their bots as proxies for concealing and protecting C2 servers and other infrastructure.

To execute this technique, threat actors purchase a domain name and then rapidly change the IP address associated with the domain so that each time a victim visits it, they are directed to a different hosting IP address. This rapid changing of IP addresses is configured *round-robin* style, a legitimate technique for balancing client requests to a web server. By shortening the time-to-live (TTL) values of the IP addresses to several minutes or even less, the threat actor creates a more elusive infrastructure, making it more difficult for network defenses to identify and block malicious traffic, and for law enforcement and other investigators to identify its full infrastructure. **Figure 13-22** breaks down what fast flux looks like in practice.

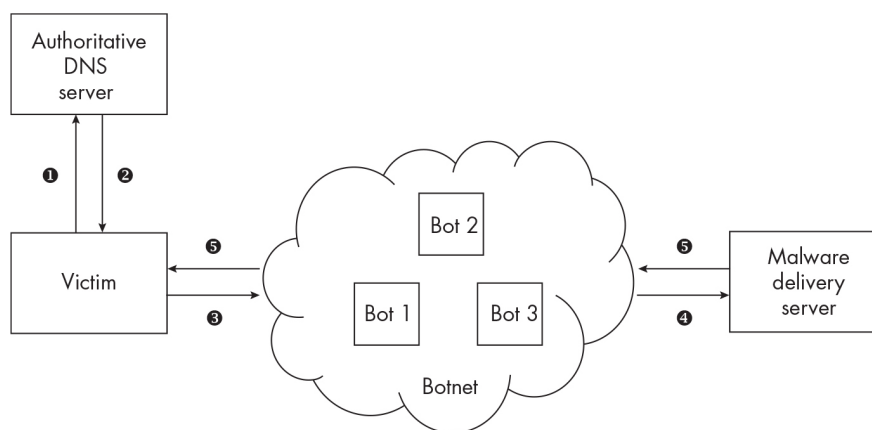


Figure 13-22: The fast-flux technique implemented in a botnet

First, a victim executes malware on their computer, and the malware wants to make an HTTP connection to the threat actor's domain, *evil.com*. Before this can happen, however, *evil.com* must be resolved to an IP address. The victim makes a DNS request ❶, and *evil.com* is resolved (by an authoritative DNS server that the malware author controls) to the IP address 59.111.180.193 ❷. Next, the malware makes an HTTP request to the IP address 59.111.180.193 ❸, which is part of a botnet and assigned to Bot 1. Acting as a proxy, Bot 1 redirects the HTTP request to the threat actor's malware delivery server ❹, which subsequently delivers the malware payload to the victim ❺.

Minutes later, a new victim (infected with the same malware) makes a DNS and HTTP request for *evil.com*, just like the first victim. This time, however, the DNS server returns 97.66.36.178 as the IP (since the threat actor has already changed the IP address associated with this domain). This IP is mapped to Bot 2, which similarly proxies and redirects the HTTP request to the malware staging server and delivers a payload to the

second victim. Because the IP address changes very frequently and is spread across the botnet (which could be a spiderweb of many thousands of systems), it is difficult to pin down the attacker's infrastructure in order to identify the other systems in the botnet and the attacker's malware distribution and C2 servers.

Fast flux is not only used in botnets; it's also used by *bulletproof hosts* (BPHs), which are web hosting providers that cater to less scrupulous web activities such as online gambling and spam or illegal activities such as organized crime. Normally, BPHs are housed in countries where this type of activity is not heavily scrutinized (and so goes unnoticed or is otherwise tolerated) and requests for takedown of malicious infrastructure from law enforcement agencies are not honored. BPHs may offer fast-flux services to cybercrime groups and others.

There's a related technique, called *double fast flux*, where not only is the IP address of the malicious domain rapidly changed, but so is the IP address of the attacker's authoritative DNS server(s). This adds an extra layer of defense and complexity for researchers and network defenses.

Very few of the techniques we have discussed throughout this chapter are effective on their own. Modern malware-based attacks often use a combination of these techniques to infiltrate their targets, as you'll see next.

Multistage and Complex Attacks

As mentioned earlier in this chapter, some modern defenses like EDR use a threat-scoring system or other analytical techniques to identify suspicious behaviors. In the early days of malware, it was common for malware to be delivered as a single malicious file. This file contained all the functionality required to infect the victim and take the further actions that the malware author intended. To successfully bypass modern endpoint and network defenses, however, malware authors must take a more complex, multistage approach. Crafting multistep, complex attack chains makes it more difficult for defenses to identify what's occurring and take appropriate action. **Figure 13-23** illustrates such an attack.

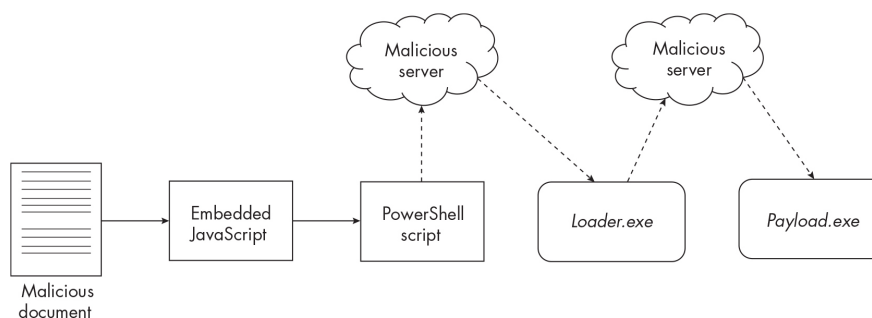


Figure 13-23: A complex, multistage attack

First, the malware author delivers a malicious document file to the victim. Once opened, the file executes embedded JavaScript code, which in turn executes obfuscated PowerShell code. The PowerShell code contacts the attacker's malware staging server to download a loader (*Loader.exe*), which is responsible for contacting another remote server to download the malware payload (*Payload.exe*).

Tracing these disparate events is more challenging for defenses. Rather than evaluating the behaviors of a single executable file, an EDR's event-tracing engine and analyzer will have to assess many different events from multiple sources before making a determination that this is malicious activity. To further complicate this attack, the malware author could spice it up with *sleep routines*. For example, the PowerShell command could wait two hours before downloading the loader executable, the loader could sleep for four hours before downloading the payload file, and then the payload could sleep for another eight hours before contacting its C2 server. The event-tracing engine would then have to account for many hours of time variance between events.

From the attacker's perspective, however, there's a downside to these multistage attacks: if even one stage of the attack fails, the entire attack chain fails. For example, if anti-malware is able to detect and quarantine the initial malicious Office document, the rest of the attack chain will fail. The same will happen if network defenses are able to identify and block the connections to the attacker's malware-staging servers. This is a gamble malware authors must take, however.

Summary

This chapter discussed many of the common techniques malware uses to actively and passively circumvent host defense controls such as anti-malware and EDR, as well as network-based controls such as NIDS. We looked in detail at how malware can enumerate a victim host to identify host defenses and how it might actively thwart those defenses. We also covered how malware can elevate its privileges, enabling it to slip past defenses more discreetly. In the next chapter, you'll learn about rootkits, a type of low-level malware that uses techniques that might be considered the ultimate form of defense evasion.